Vincent Barichard
vincent.barichard@univ-angers.fr

Constraint Handling Rules (CHR)

Une introduction à la programmation logique par règles

LERIA – Université d'Angers

05 Nov 2025



Table des matières

1 - Démarrons par un exemple
2 - Les règles de simplification
3 - Les règles de simpagation
4 - Les règles de propagation
5 - Ajouter de l'information aux contraintes
6 - Conditionner le déclenchement des règles
7 - CHR et les variables logiques
8 - Synthèse de cette introduction à CHR
9 - Quel système CHR choisir ?

- Beaucoup des exemples présentés dans ces transparents sont issus de présentations et de tutoriels réalisés par *Thom Frühwirth* (créateur de CHR) et *Tom Schrijvers*
- Les exemples simples sont exécutés avec `CHR.js` :
  - http://chrjs.net/playground.html (*Christian Holzbaur*)
- Les exemples plus complexes sont exécutés avec CHR++ :
  - https://gitlab.com/vynce/chrpp (*Vincent Barichard*)

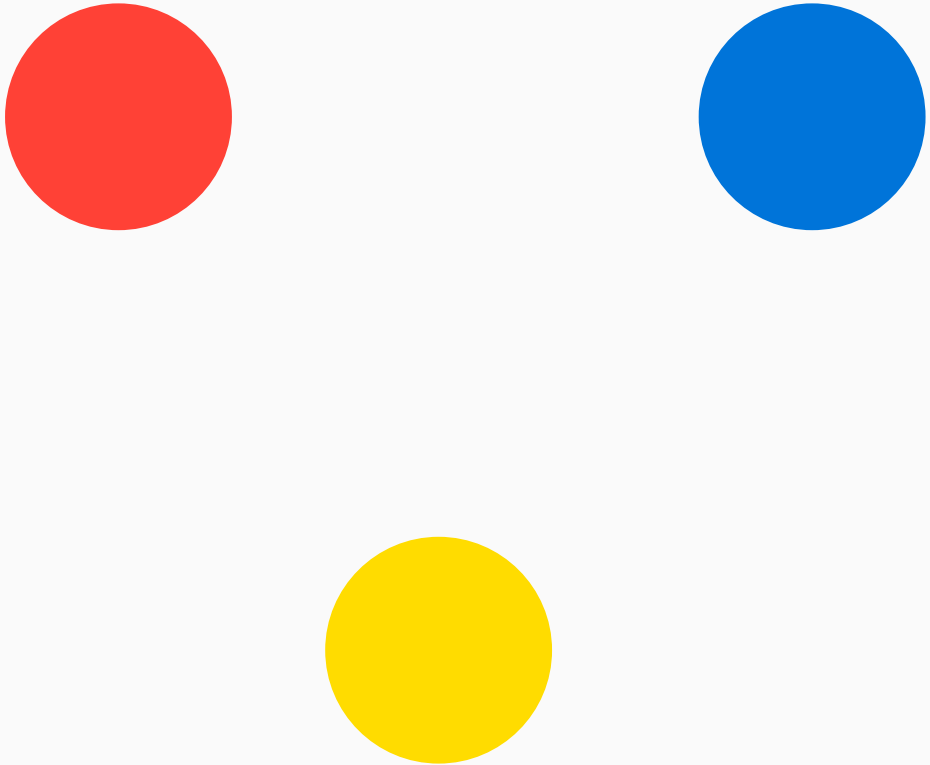
1 - Démarrons par un exemple

Le mélange des couleurs

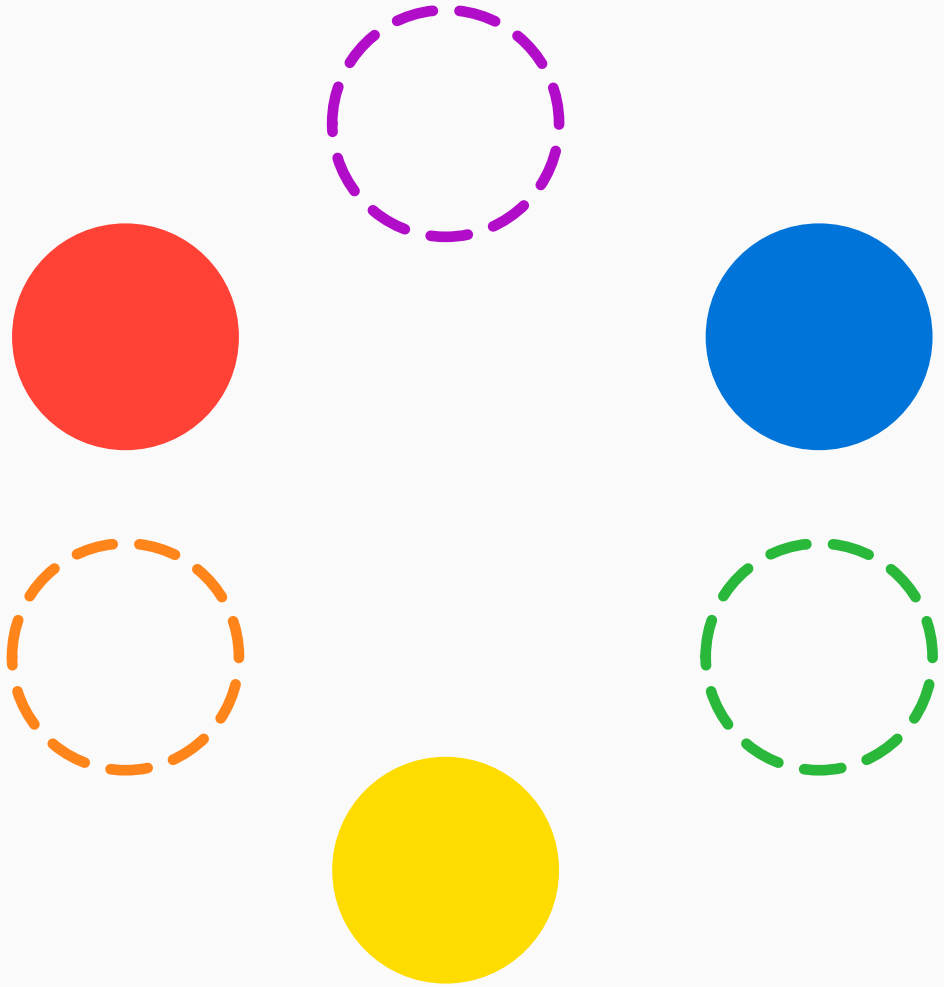
Tout jeune écolier apprend que le mélange de certaines couleurs primaires permet d'obtenir d'autres couleurs. Par exemple, le mélange du rouge et du bleu donne du violet. Ainsi il doit :

- apprendre comment obtenir de nouvelles couleurs ;
- en mélangeant des couleurs déjà connues ;
- un écolier avisé se facilitera les efforts futurs en écrivant un petit langage de règles de mélange de couleurs (RMC).

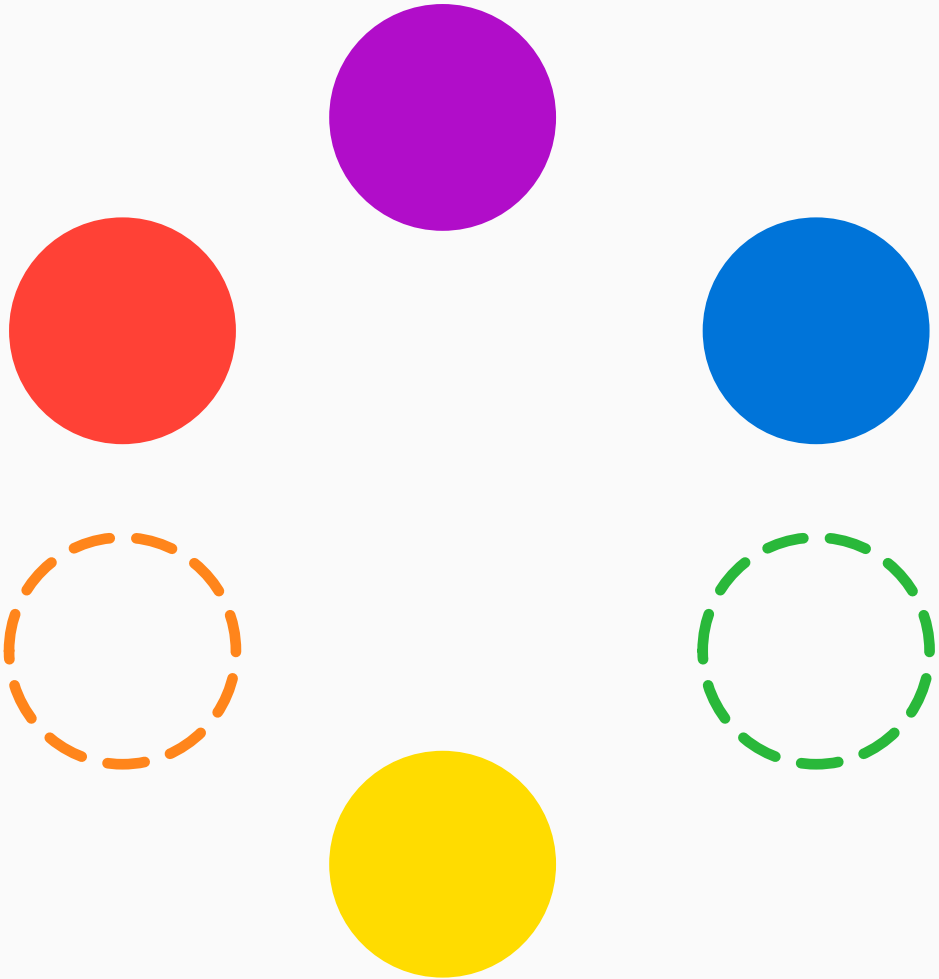
La palette de couleurs

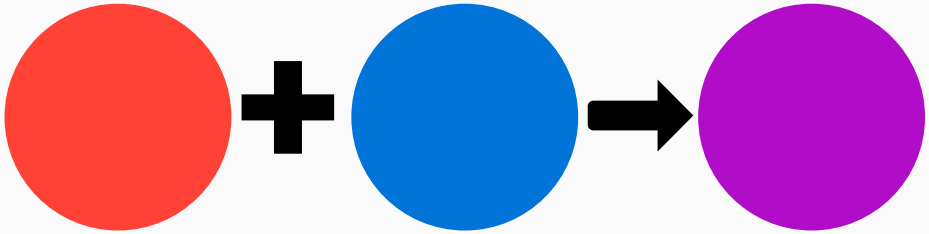


La palette de couleurs

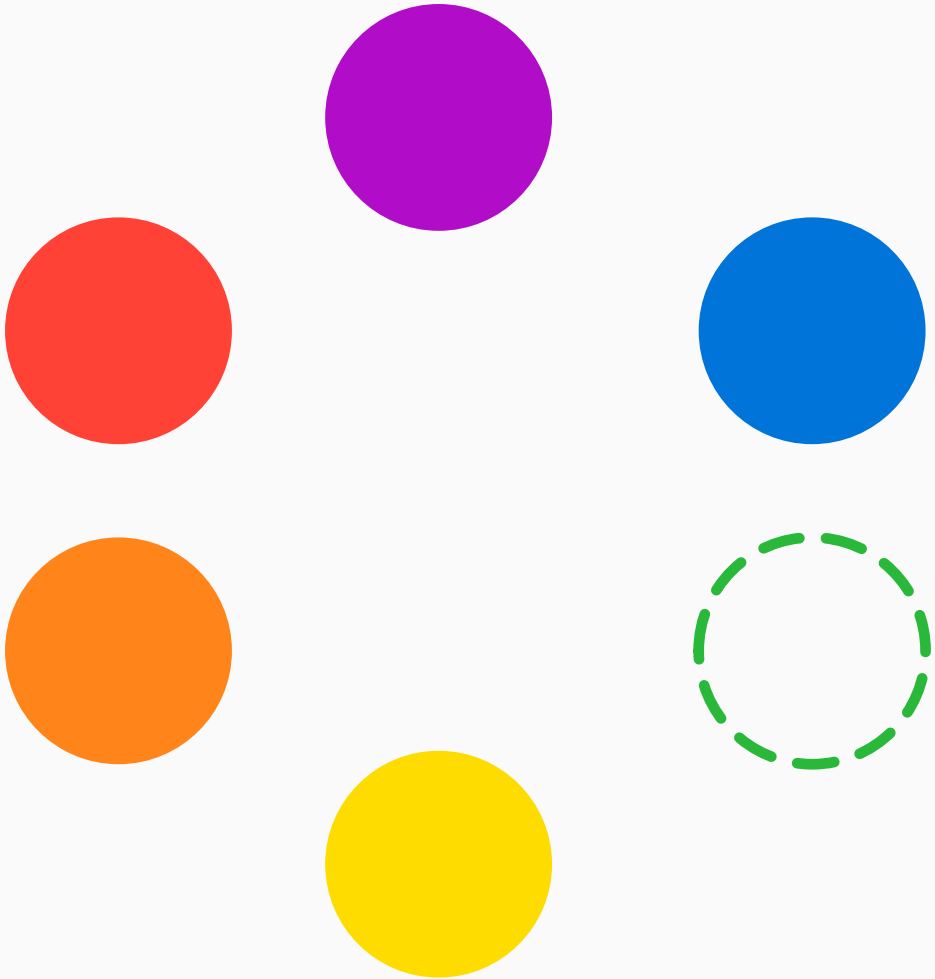


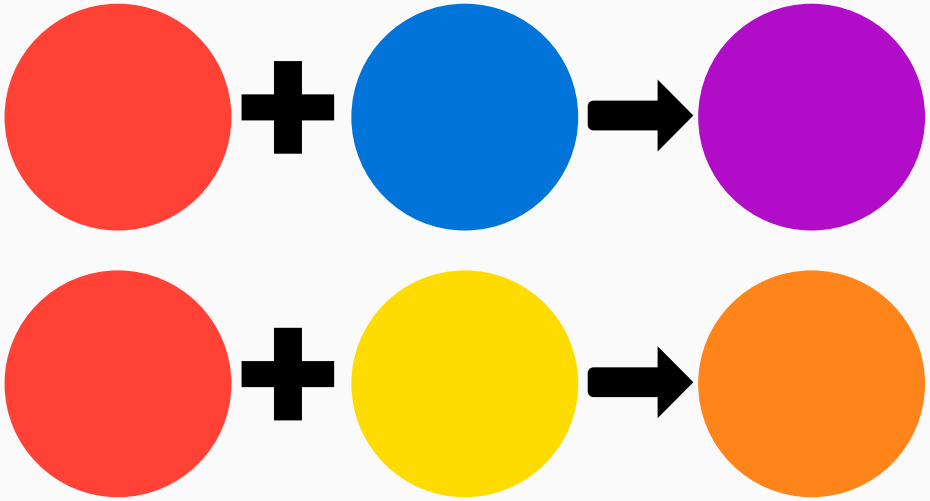
La palette de couleurs



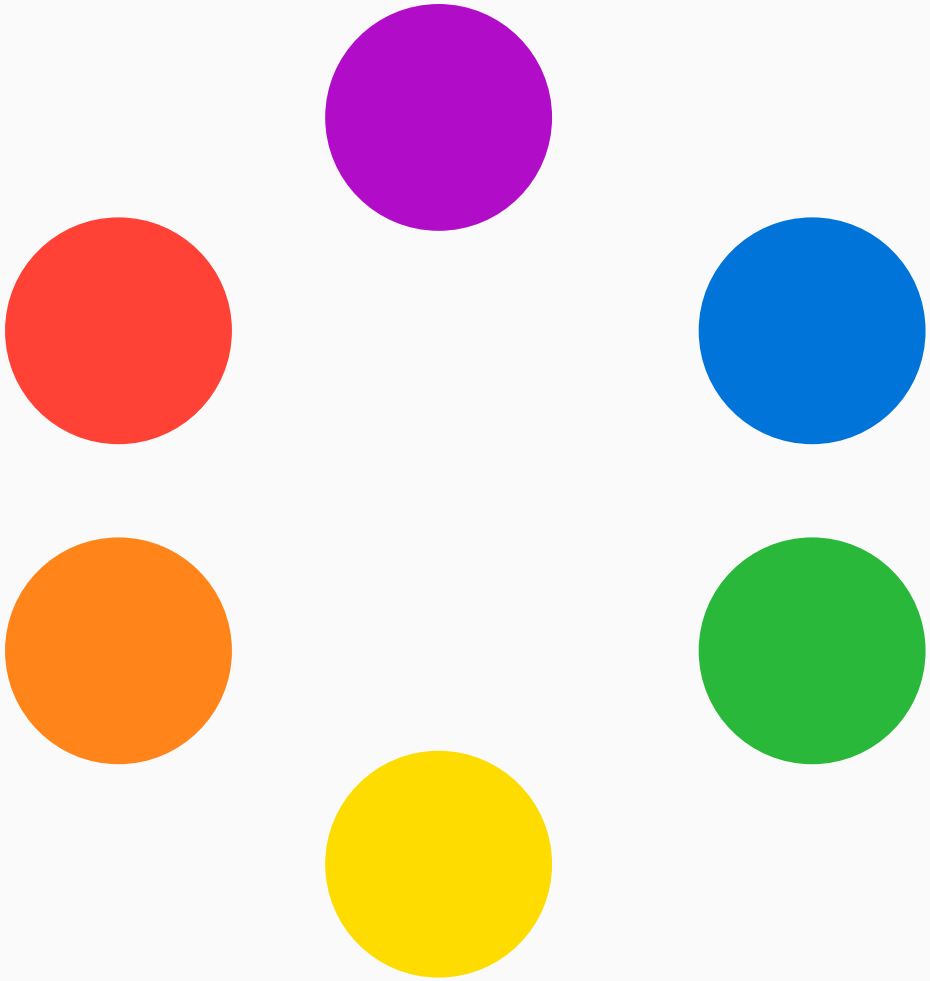


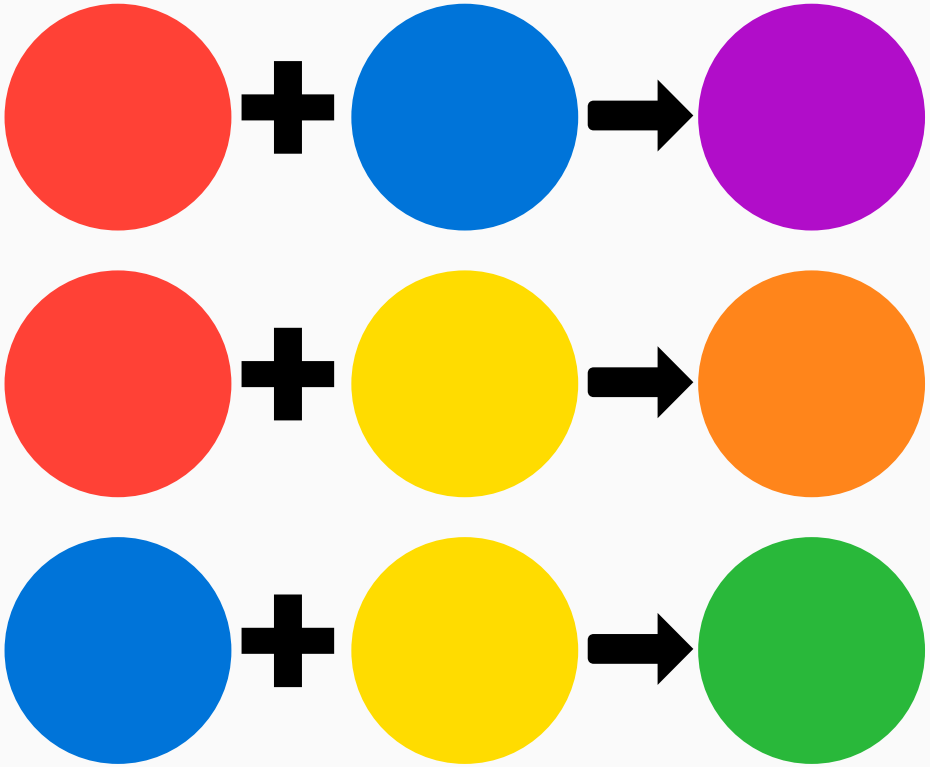
La palette de couleurs





La palette de couleurs





2 - Les règles de simplification

Traduction de RMC en CHR



Les concepts clés de RMC peuvent être traduits en concepts équivalents dans CHR.

RMC		CHR
couleurs	➡	contraintes
règles	➡	règles de simplication
mélange de couleurs	➡	*store* de contraintes

Couleurs vers contraintes

Déclarons nos couleurs comme des contraintes CHR :

CHR (CHR++)

```
<chr_constraints> red()
```



Couleurs vers contraintes

Déclarons nos couleurs comme des contraintes CHR :

CHR (CHR++)

```
<chr_constraints> red()
<chr_constraints> blue()
```



Couleurs vers contraintes

Déclarons nos couleurs comme des contraintes CHR :

CHR (CHR++)

```
<chr_constraints> red()
<chr_constraints> blue()
<chr_constraints> yellow()
```



Couleurs vers contraintes

Déclarons nos couleurs comme des contraintes CHR :

CHR (CHR++)

```
<chr_constraints> red()
<chr_constraints> blue()
<chr_constraints> yellow()
<chr_constraints> purple()
<chr_constraints> orange()
<chr_constraints> green()
…
```



Couleurs vers contraintes

Les couleurs sont-elles des contraintes ?

- Non, pas dans le sens de la programmation par contraintes.
- Oui, dans le sens de la programmation logique par règles. Ce sont prédicats du première ordre réécrits par application de règles.

Règles vers contraintes

Règle de simplication *simplifiée* CHR

Une règle de simplication CHR est une règle de réécriture de la forme :

Head <=> Body

Si les éléments de la tête sont présents, alors les éléments de la tête sont remplacés par ceux du corps.

♠

Règles vers contraintes

Règle de simplication *simplifiée* CHR

Une règle de simplication CHR est une règle de réécriture de la forme :

Head <=> Body

Si les éléments de la tête sont présents, alors les éléments de la tête sont remplacés par ceux du corps.

♠

CHR (CHR.js)



```
red, blue <=> purple
```

Règles vers contraintes

Règle de simplication *simplifiée* CHR

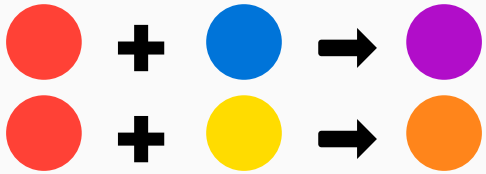
Une règle de simplication CHR est une règle de réécriture de la forme :

Head <=> Body

Si les éléments de la tête sont présents, alors les éléments de la tête sont remplacés par ceux du corps.

♠

CHR (CHR.js)



```
red, blue <=> purple
red, yellow <=> orange
```

Règles vers contraintes

Règle de simplication *simplifiée* CHR

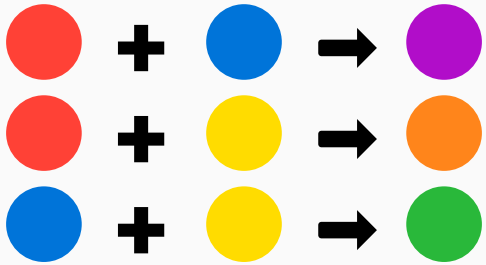
Une règle de simplication CHR est une règle de réécriture de la forme :

Head <=> Body

Si les éléments de la tête sont présents, alors les éléments de la tête sont remplacés par ceux du corps.

♠

CHR (CHR.js)



```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

Mise en application

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

Questions / Réponses (CHR.js)

? - red, blue

? - red, yellow ?

? - blue, yellow ?

Mise en application

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

Questions / Réponses (CHR.js)

? - red, blue
purple

? - blue, yellow ?

? - red, yellow ?

Mise en application

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

Questions / Réponses (CHR.js)

? - red, blue
purple

? - blue, yellow ?
green

? - red, yellow ?

Mise en application

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

Questions / Réponses (CHR.js)

? - red, blue
purple

? - blue, yellow ?
green

? - red, yellow ?
orange

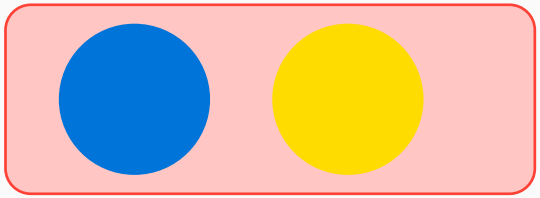
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?





› Les contraintes sont traitées de gauche à droite.

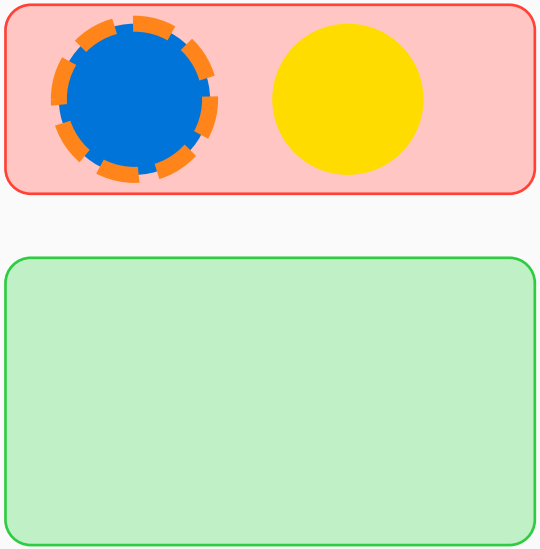
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



- Les contraintes sont traitées de gauche à droite.
- Activation de la contrainte blue.

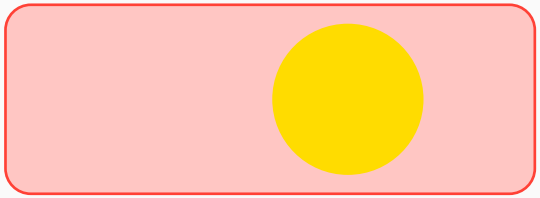
Déroulement de l'exécution du programme

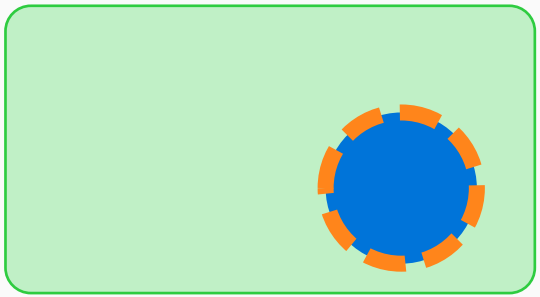
Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?





- Les contraintes sont traitées de gauche à droite.
- Activation de la contrainte blue.
- Ajout de la contrainte blue au *store*.

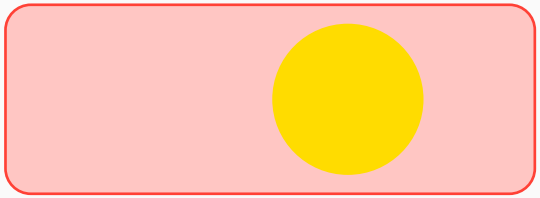
Déroulement de l'exécution du programme

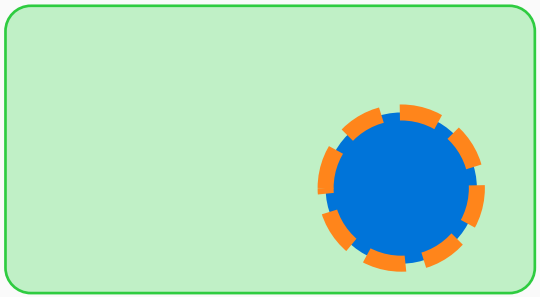
Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?





- Les contraintes sont traitées de gauche à droite.
- Activation de la contrainte blue.
- Ajout de la contrainte blue au *store*.
- Déclenchement d'une règle ?
  - *Non*, aucun partenaire red dans le *store*.

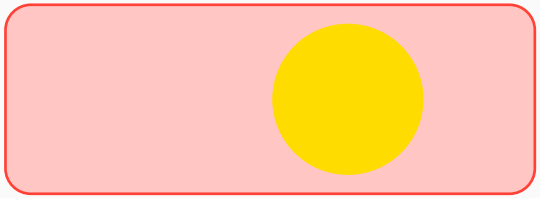
Déroulement de l'exécution du programme

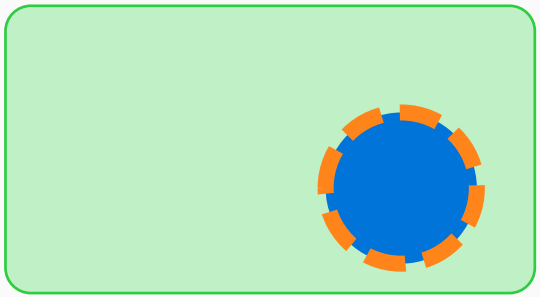
Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?





- Les contraintes sont traitées de gauche à droite.
- Activation de la contrainte blue.
- Ajout de la contrainte blue au *store*.
- Déclenchement d'une règle ?
  - *Non*, aucune contrainte blue dans la tête.

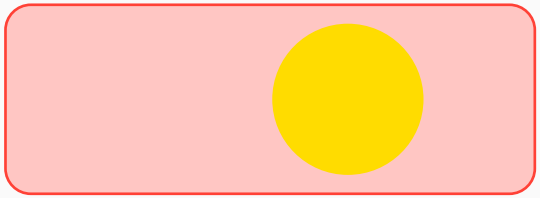
Déroulement de l'exécution du programme

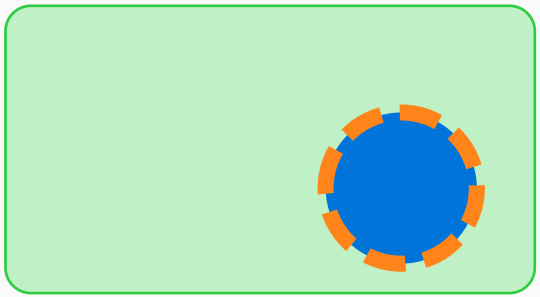
Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?





- Les contraintes sont traitées de gauche à droite.
- Activation de la contrainte blue.
- Ajout de la contrainte blue au *store*.
- Déclenchement d'une règle ?
  - *Non*, aucun partenaire yellow dans le *store*.

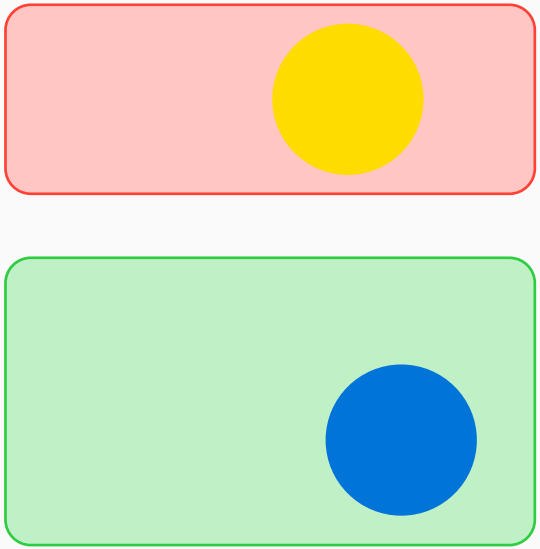
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



- Les contraintes sont traitées de gauche à droite.
- Activation de la contrainte blue.
- Ajout de la contrainte blue au *store*.
- Déclenchement d'une règle ?
  - *Non*, aucun partenaire yellow dans le *store*.
- Désactivation de la contrainte.

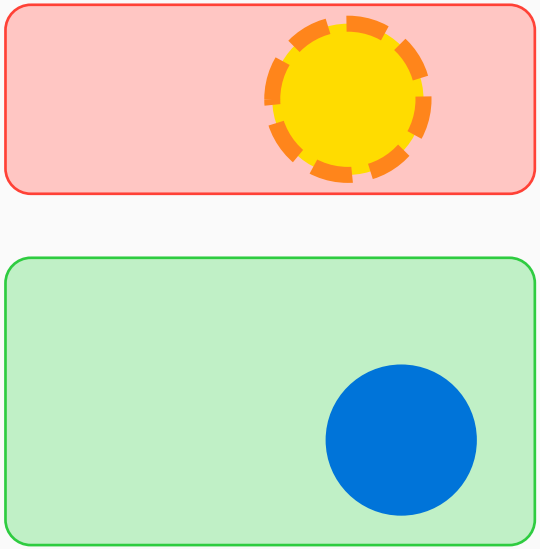
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



› Activation de la contrainte yellow.

Déroulement de l'exécution du programme

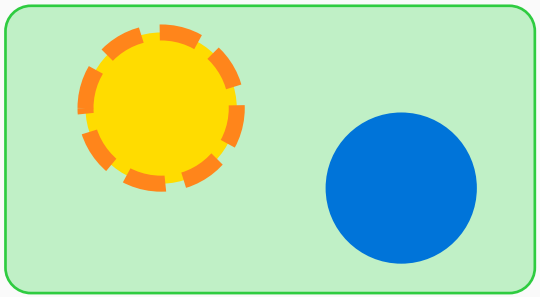
Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?





- Activation de la contrainte yellow.
- Ajout de la contrainte yellow au *store*.

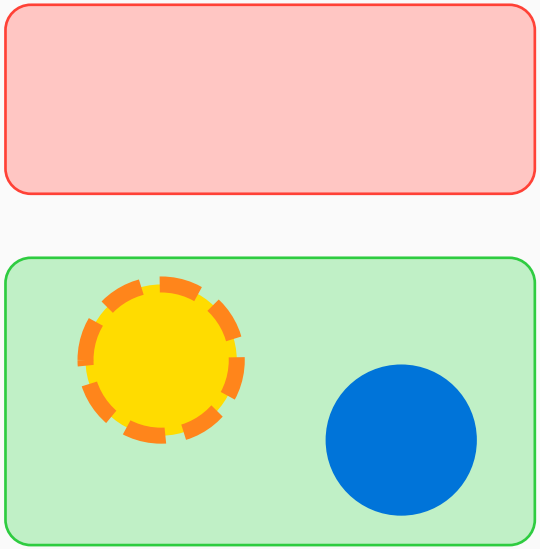
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



- Activation de la contrainte yellow.
- Ajout de la contrainte yellow au *store*.
- Déclenchement d'une règle ?
  - *Non*, aucune contrainte yellow dans la tête.

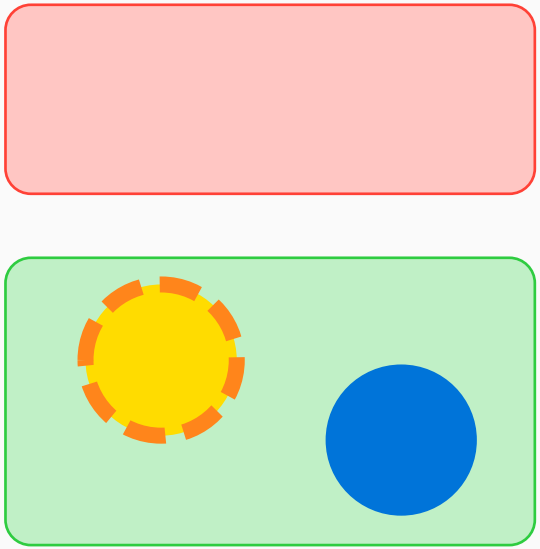
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



- Activation de la contrainte yellow.
- Ajout de la contrainte yellow au *store*.
- Déclenchement d'une règle ?
  - *Non*, aucun partenaire red dans le *store*.

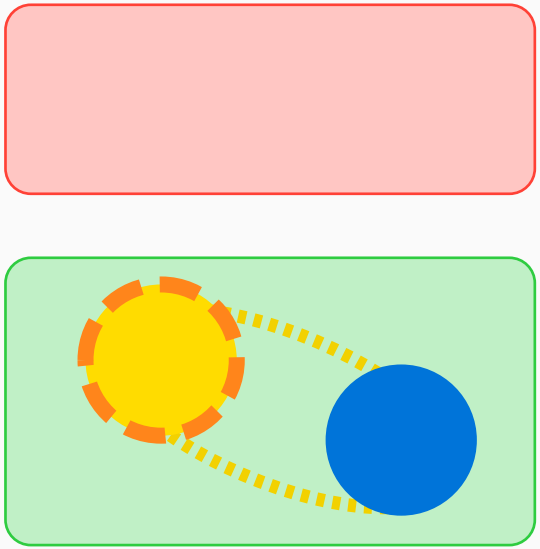
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



- Activation de la contrainte yellow.
- Ajout de la contrainte yellow au *store*.
- Déclenchement d'une règle ?
  - *Oui*, avec la contrainte blue comme partenaire.

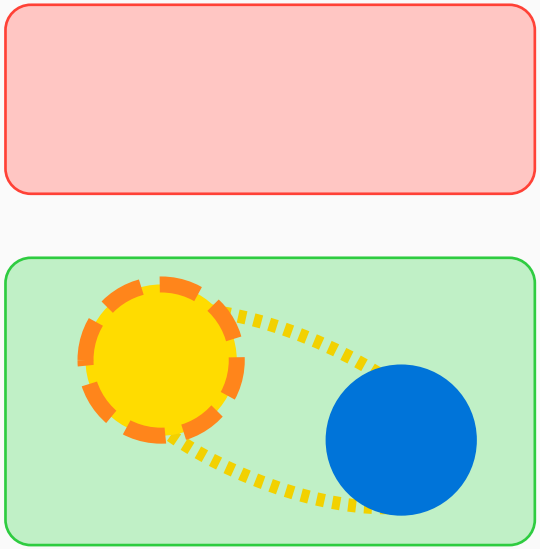
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



- Activation de la contrainte yellow.
- Ajout de la contrainte yellow au *store*.
- Déclenchement d'une règle ?
  - *Oui*, avec la contrainte blue comme partenaire.
- Exécution de la règle !

Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?





- Exécution de la règle !
- Suppression des contraintes de la tête.

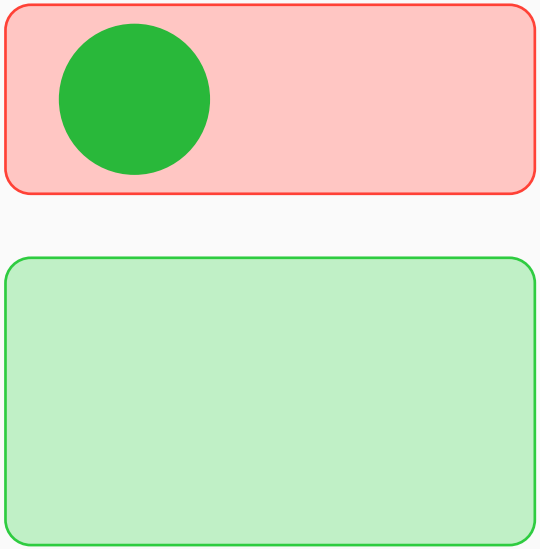
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



- Exécution de la règle !
- Suppression des contraintes de la tête.
- Ajout de la contrainte green au début du but.

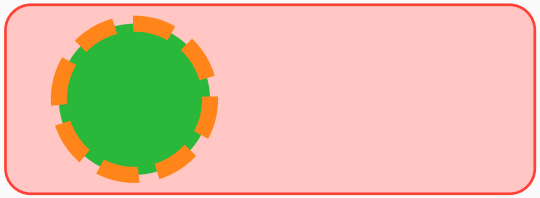
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



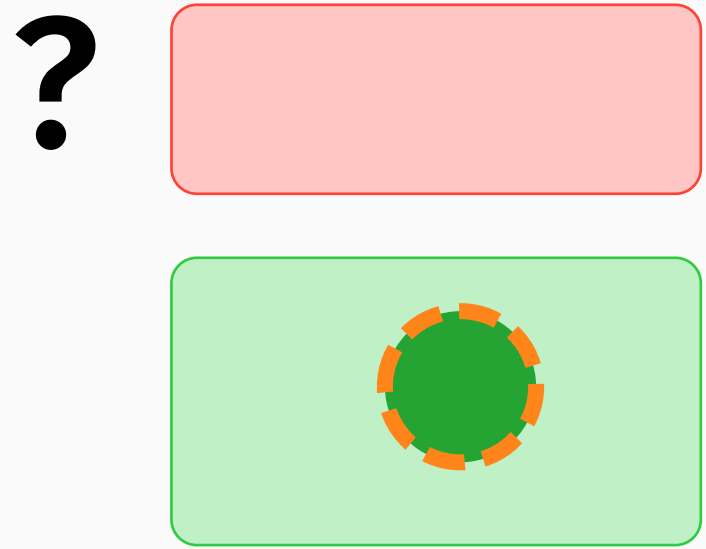


- Exécution de la règle !
- Suppression des contraintes de la tête.
- Ajout de la contrainte green au début du but.
- Activation de la contrainte green.

Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```



- Exécution de la règle !
- Suppression des contraintes de la tête.
- Ajout de la contrainte green au début du but.
- Activation de la contrainte green.
- Ajout de la contrainte green au *store*.

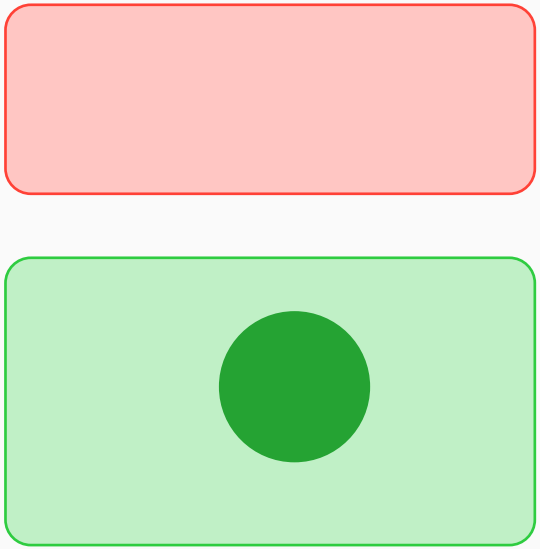
Déroulement de l'exécution du programme

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

?



- Exécution de la règle !
- Suppression des contraintes de la tête.
- Ajout de la contrainte green au début du but.
- Activation de la contrainte green.
- Ajout de la contrainte green au *store*.
- ... et finalement il reste green dans le *store*.

Quelle est la sémantique adoptée par CHR ?

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

Questions / Réponses

› ? - yellow, red
» orange
› ? - yellow, red, blue
» orange, blue

Quelle est la sémantique adoptée par CHR ?

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

Questions / Réponses

› ? - yellow, red
» orange
› ? - yellow, red, blue
» orange, blue *Et pourquoi pas yellow, purple ?*

Sémantique abstraite et sémantique raffinée

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

♠

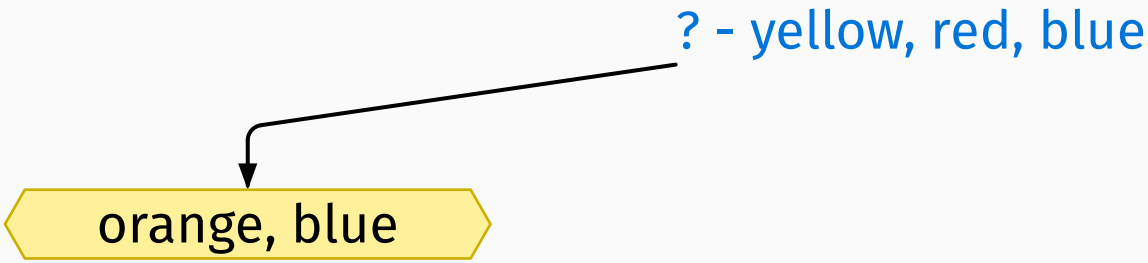
? - yellow, red, blue

Sémantique abstraite et sémantique raffinée

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```

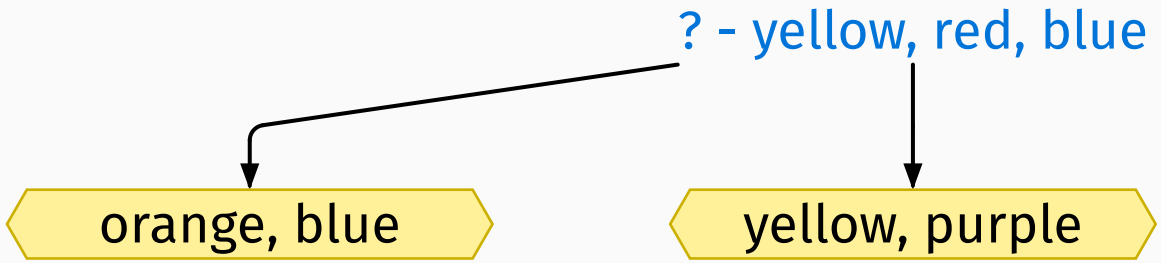
♠



Sémantique abstraite et sémantique raffinée

Programme (CHR.js)

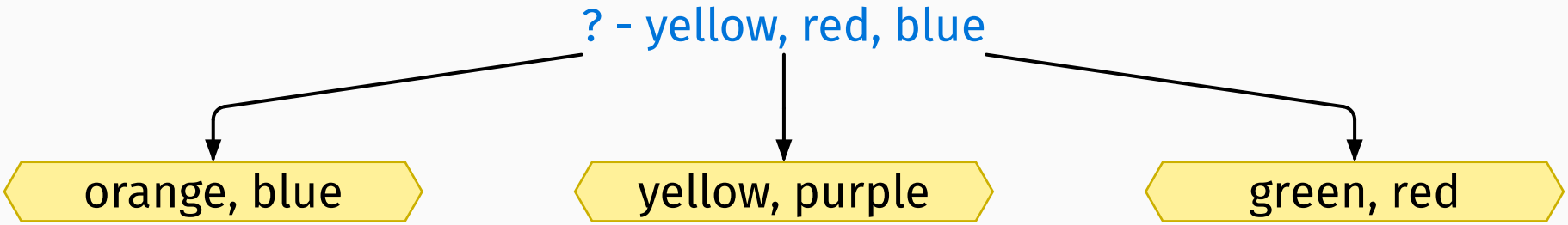
```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```



Sémantique abstraite et sémantique raffinée

Programme (CHR.js)

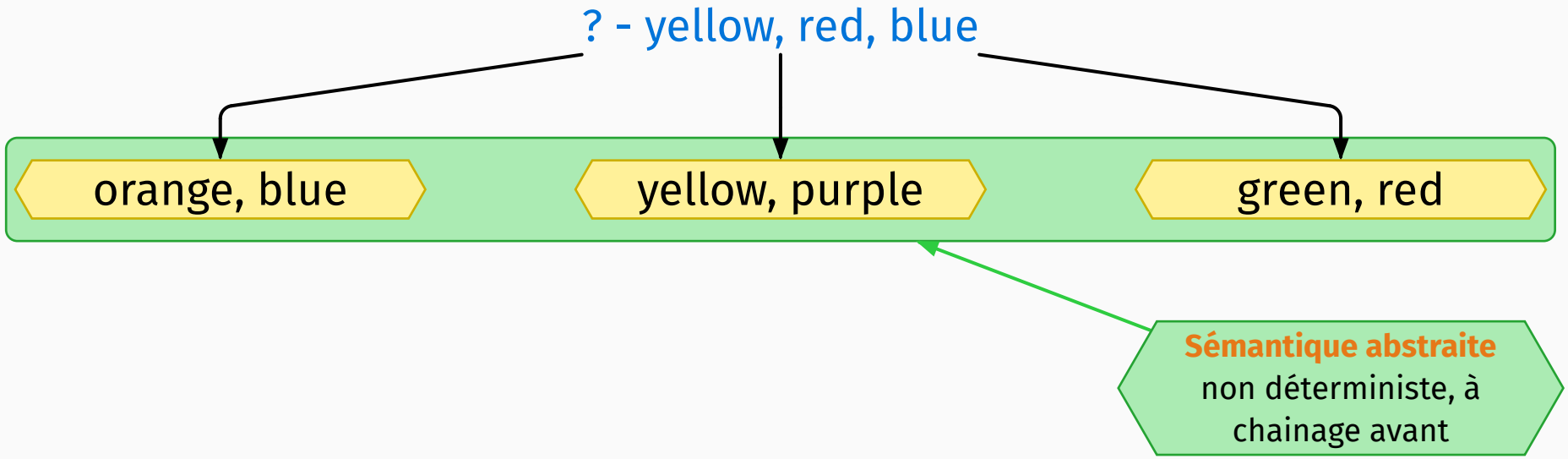
```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```



Sémantique abstraite et sémantique raffinée

Programme (CHR.js)

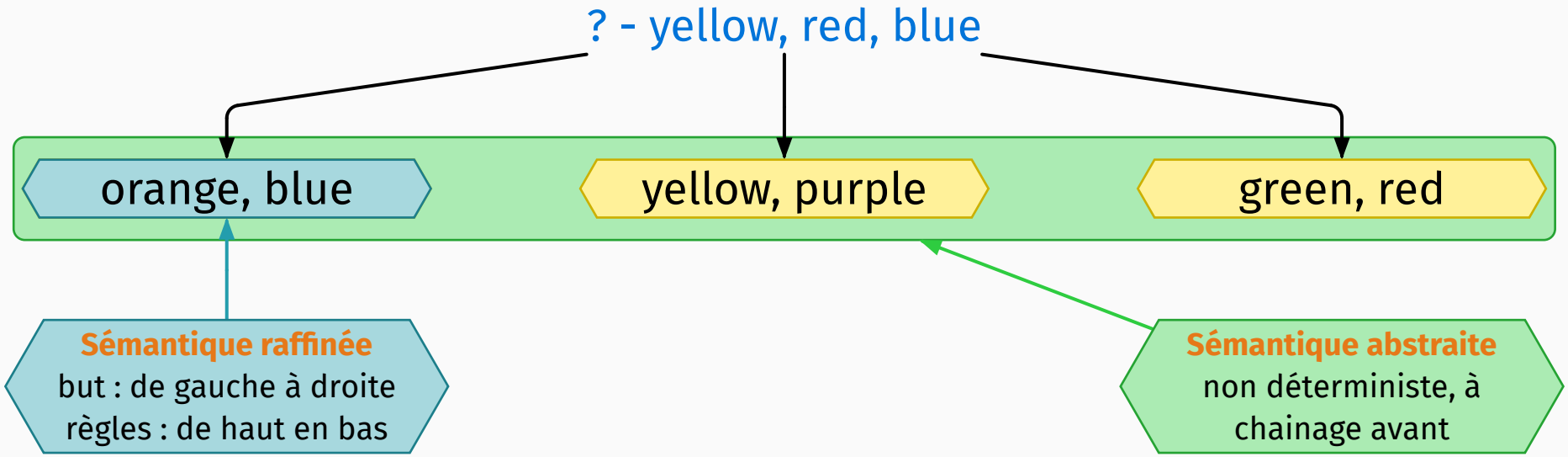
```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```



Sémantique abstraite et sémantique raffinée

Programme (CHR.js)

```
red, blue <=> purple
red, yellow <=> orange
blue, yellow <=> green
```



En résumé

Règle de simplification

- *head <=> Body*
- Le corps remplace la tête

Deux niveaux d'exécution

- Sémantique abstraite (haut niveau) :
  - Non déterministe, à chainage arrière.
- Sémantique raffinée (bas niveau) :
  - déterministe, toujours à chainage avant ;
  - but traité de gauche à droite ;
  - règles analysées du haut vers le bas.

3 - Les règles de simpagation

Ajoutons une couleur absorbante

Règle de simpagation *simplifiée* CHR

$Head_k \setminus Head_r \Leftrightarrow Body$

$Head_r$ est remplacée par *Body* lorsque $Head_k$ est présent.

♠

Ajoutons une couleur absorbante

Règle de simpagation *simplifiée* CHR

$Head_k \setminus Head_r \Leftrightarrow Body$

$Head_r$ est remplacée par $Body$ lorsque $Head_k$ est présent.

♠

CHR (`CHR.js`)

› Marron reste marron ...



```
brown \ yellow <=> true
```

Ajoutons une couleur absorbante

Règle de simpagation *simplifiée* CHR

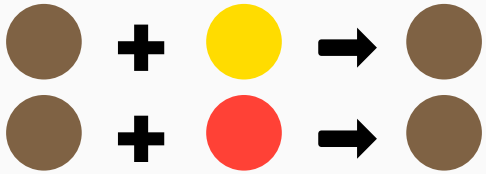
$$Head_k \setminus Head_r \Leftrightarrow Body$$

$Head_r$ est remplacée par $Body$ lorsque $Head_k$ est présent.

♠

CHR (`CHR.js`)

- Marron reste marron ...



```
brown \ yellow <=> true
brown \ red <=> true
```

Ajoutons une couleur absorbante

Règle de simpagation *simplifiée* CHR

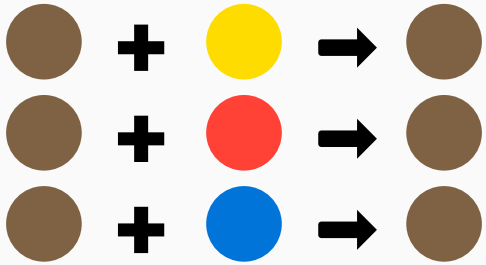
$$Head_k \setminus Head_r \Leftrightarrow Body$$

$Head_r$ est remplacée par $Body$ lorsque $Head_k$ est présent.

♠

CHR (`CHR.js`)

› Marron reste marron ...



```
brown \ yellow <=> true
brown \ red <=> true
brown \ blue <=> true
```

Transmutation du plomb en or !

En alchimie ...

```
pierre_philosophale \ plomb <=> or
```

♠

Transmutation du plomb en or !

En alchimie ...

```
pierre_philosophale \ plomb <=> or
```

♠

CHR (`CHR.js`)

- pierre_philosophale, plomb »
- plomb, plomb »
- pierre_philosophale, plomb, plomb »
- plomb, plomb, pierre_philosophale »

Transmutation du plomb en or !

En alchimie ...

```
pierre_philosophale \ plomb <=> or
```

♠

CHR (`CHR.js`)

- pierre_philosophale, plomb
  - » pierre_philosophale, or
- pierre_philosophale, plomb, plomb
  - »
- plomb, plomb
  - »
- plomb, plomb, pierre_philosophale
  - »

Transmutation du plomb en or !

En alchimie ...

```
pierre_philosophale \ plomb <=> or
```

♠

CHR (CHR.js)

- pierre_philosophale, plomb
  - » pierre_philosophale, or
- pierre_philosophale, plomb, plomb
  - » pierre_philosophale, or, or
- plomb, plomb
  - »
- plomb, plomb, pierre_philosophale
  - »

Transmutation du plomb en or !

En alchimie ...

```
pierre_philosophale \ plomb <=> or
```

♠

CHR (CHR.js)

- pierre_philosophale, plomb
  - » pierre_philosophale, or
- pierre_philosophale, plomb, plomb
  - » pierre_philosophale, or, or
- plomb, plomb
  - » plomb, plomb
- plomb, plomb, pierre_philosophale
  - »

Transmutation du plomb en or !

En alchimie ...

```
pierre_philosophale \ plomb <=> or
```

♠

CHR (`CHR.js`)

- pierre_philosophale, plomb
  - » pierre_philosophale, or
- pierre_philosophale, plomb, plomb
  - » pierre_philosophale, or, or
- plomb, plomb
  - » plomb, plomb
- plomb, plomb, pierre_philosophale
  - » pierre_philosophale, or, or

Transmutation du plomb en or !

En alchimie ...

```
pierre_philosophale \ plomb <=> or
```

♠

CHR (`CHR.js`)

- pierre_philosophale, plomb
  - » pierre_philosophale, or
- plomb, plomb
  - » plomb, plomb
- pierre_philosophale, plomb, plomb
  - » pierre_philosophale, or, or
- plomb, plomb, pierre_philosophale
  - » pierre_philosophale, or, or

Important

Les règles sont déclenchées jusqu'à épuisement des possibilités.

En résumé

Règles

- *head* <=> *Body*
  - Le corps remplace la tête.
- $Head_k \setminus Head_r$ <=> *Body*
  - Le corps remplace $Head_r$ si $Head_k$ est présent.

Deux niveaux d'exécution

- Les règles sont déclenchées jusqu'à épuisement des possibilités.
- Le *store* de contraintes est un multiensemble, une contrainte peut apparaître plusieurs fois.

4 - Les règles de propagation

Et si je voulais conserver les couleurs initiales ?

Programme (CHR.js)

```
red, blue ==> purple
red, yellow ==> orange
blue, yellow ==> green
```

♠

Et si je voulais conserver les couleurs initiales ?

Programme (CHR.js)

```
red, blue ==> purple
red, yellow ==> orange
blue, yellow ==> green
```

♠

CHR (CHR.js)

› red, yellow
»
› red, yellow, blue
»
› red, yellow, blue, yellow
»

Et si je voulais conserver les couleurs initiales ?

Programme (CHR.js)

```
red, blue ==> purple
red, yellow ==> orange
blue, yellow ==> green
```

♠

CHR (CHR.js)

› red, yellow
  » red, yellow, orange
› red, yellow, blue
  »
› red, yellow, blue, yellow
  »

Et si je voulais conserver les couleurs initiales ?

Programme (CHR.js)

```
red, blue ==> purple
red, yellow ==> orange
blue, yellow ==> green
```

♠

CHR (CHR.js)

› red, yellow
  » red, yellow, orange
› red, yellow, blue
  » red, yellow, orange, blue, purple, green
› red, yellow, blue, yellow
  »

Et si je voulais conserver les couleurs initiales ?

Programme (CHR.js)

```
red, blue ==> purple
red, yellow ==> orange
blue, yellow ==> green
```

♠

CHR (CHR.js)

- red, yellow
  - » red, yellow, orange
- red, yellow, blue
  - » red, yellow, orange, blue, purple, green
- red, yellow, blue, yellow
  - » red, yellow, orange, blue, purple, green, yellow, orange, green

Règles de propagation

Règle de propagation *simplifiée* CHR

Head ==> Body

Le corps est ajouté lorsque la tête est présente.

♠

Règles de propagation

Règle de propagation *simplifiée* CHR

Head ==> Body

Le corps est ajouté lorsque la tête est présente. ♠

- Une règle de propagation ne peut se déclencher qu'une seule fois pour une même instance de la tête (même ensemble de contraintes).
- Un historique des applications de chaque règle de propagation est maintenu pour éviter les redéclenchements.

Attention aux récursions !

Attention aux récursions !

```
red, blue ==> red
```
♠

- La règle se déclenche indéfiniment.
- Les règles de simpagation permettent d'éviter les problèmes de terminaisons triviales.

En résumé

Règles

- *head <=> Body*
  - Le corps remplace la tête.
- $Head_k$ \ $Head_r$ *<=> Body*
  - Le corps remplace $Head_r$ si $Head_k$ est présent.
- *head ==> Body*
  - Le corps est ajouté lorsque la tête est présente.

5 - Ajouter de l’information aux contraintes

Économisons ...

Programme (CHR.js)

```
tirelire(I), tirelire(J) <=> tirelire(I+J)
```

♠

?- tirelire(5 €), tirelire(1 €), tirelire(4 €), tirelire(2 €)

Économisons ...

Programme (CHR.js)

```
tirelire(I), tirelire(J) <=> tirelire(I+J)
```



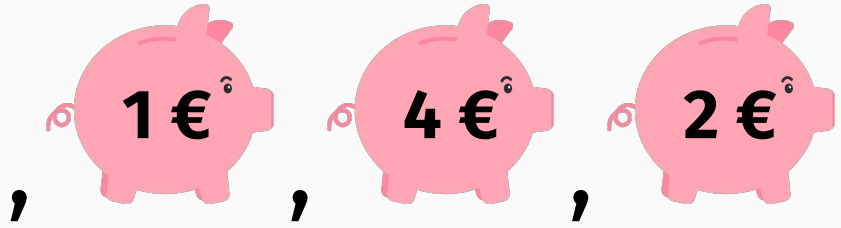
Économisons ...

Programme (CHR.js)

```
tirelire(I), tirelire(J) <=> tirelire(I+J)
```

♠

?-



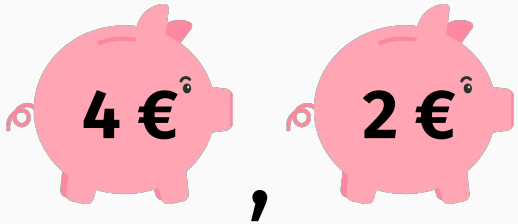


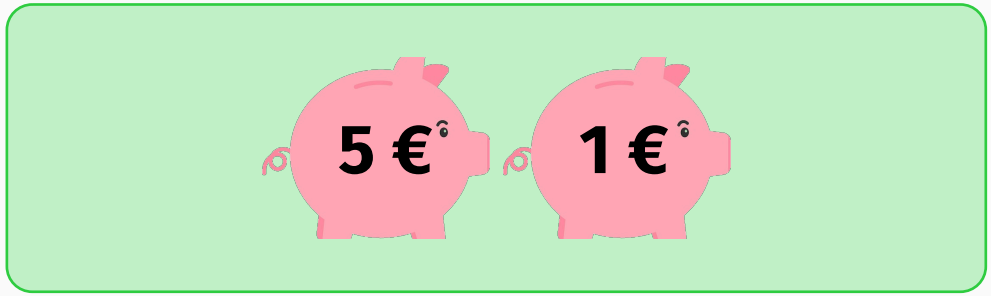
Économisons ...

Programme (CHR.js)

```
tirelire(I), tirelire(J) <=> tirelire(I+J)
```

♠

?- , , 

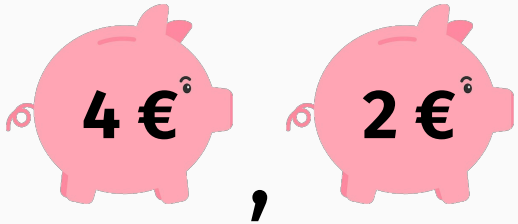


Économisons ...

Programme (CHR.js)

```
tirelire(I), tirelire(J) <=> tirelire(I+J)
```

♠

?- , , 

,



Économisons ...

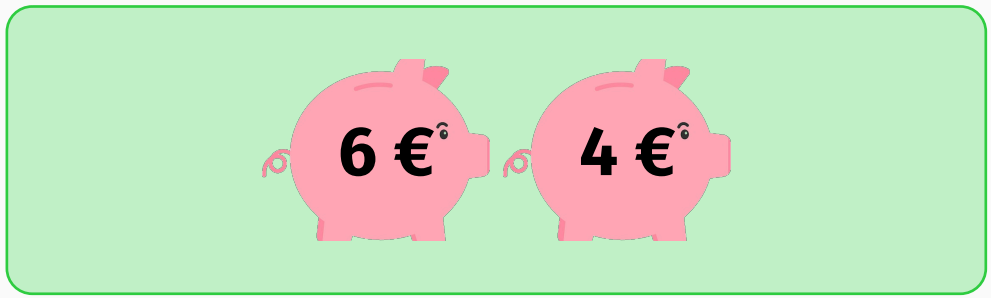
Programme (CHR.js)

```
tirelire(I), tirelire(J) <=> tirelire(I+J)
```

♠

?- , , ,





Économisons ...

Programme (CHR.js)

```
tirelire(I), tirelire(J) <=> tirelire(I+J)
```

♠

?- , , ,





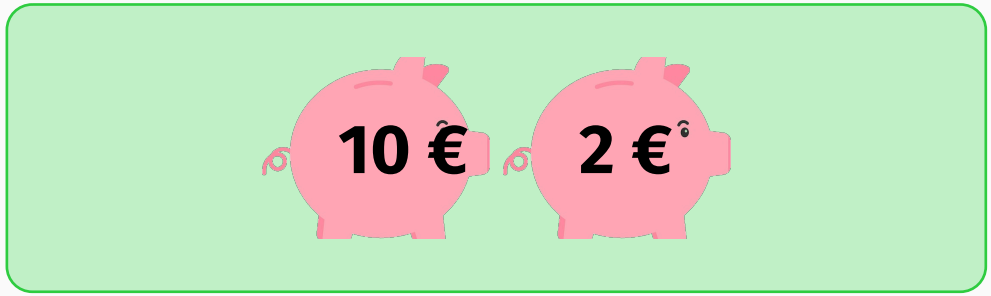
Économisons ...

Programme (CHR.js)

```
tirelire(I), tirelire(J) <=> tirelire(I+J)
```

♠

?- , , ,



Économisons ...

Programme (CHR.js)

```
tirelire(I), tirelire(J) <=> tirelire(I+J)
```

♠

?- , , ,



Intégration de CHR dans un langage hôte

- Particularité de CHR : nécessite d'être hébergé par un langage hôte.
- Communication bidirectionnelle entre le langage hôte et CHR :
  - le langage hôte ajoute des contraintes au *store* ;
  - Le corps des règles fait appel aux instructions du langage hôte ;
  - Utilisation des opérateurs arithmétiques du langage hôte : I+J.
- Les arguments ne sont pas limités aux entiers, mais aux structures de données du langage hôte.

6 - Conditionner le déclenchement des règles

L'ajout d'une garde

- Modifions notre programme pour séparer les centimes des euros.

Programme (CHR.js)

```
piece(I), tirelire(J) <=> I >= 1 | tirelire(I+J)
piece(I), tirelire_centimes(J) <=> I < 1 | tirelire_centimes(I+J)
```

L'ajout d'une garde

- Modifions notre programme pour séparer les centimes des euros.

Programme (CHR.js)

```
piece(I), tirelire(J) <=> I >= 1 | tirelire(I+J)
piece(I), tirelire_centimes(J) <=> I < 1 | tirelire_centimes(I+J)
```

♠

CHR (CHR.js)

- tirelire(0), tirelire_centimes(0), piece(2), piece(1), piece(1), piece(0.5), piece(0.1)
- tirelire(4), tirelire_centimes(0.6)

L'ajout d'une garde

- Modifions notre programme pour séparer les centimes des euros.

Programme (CHR.js)

```
piece(I), tirelire(J) <=> I >= 1 | tirelire(I+J)
piece(I), tirelire_centimes(J) <=> I < 1 | tirelire_centimes(I+J)
```

♠

CHR (CHR.js)

- tirelire(0), tirelire_centimes(0), piece(2), piece(1), piece(1), piece(0.5), piece(0.1)
- tirelire(4), tirelire_centimes(0.6)

- La garde conditionne le déclenchement de la règle
  - L'ordre des règles étant fixé par la sémantique, la deuxième garde est inutile.

Exercice : algorithme de calcul du minimum

Programme (CHR.js)

```
Une contrainte : min(+int) avec un argument entier.

Quelle(s) règle(s) écrire pour qu'il ne reste que la contrainte min
dont l'argument est celui de plus petite valeur ?
```

♠

Exécution recherchée

› min(4), min(2), min(3), min(1), min(5)
  » min(1)

Factorisons le programme de mélange des couleurs

- Réécrivons notre programme de mélange des couleurs avec des contraintes avec arguments.
  - L'argument de `color()` est ici une constante nommée associée à un entier.

Programme (CHR.js)

```
color(X), color(Y) <=> X == red, Y == blue | color(purple)
color(X), color(Y) <=> X == red, Y == yellow | color(orange)
color(X), color(Y) <=> X == blue, Y == yellow | color(green)
```

Factorisons le programme de mélange des couleurs

- Réécrivons notre programme de mélange des couleurs avec des contraintes avec arguments.
  - L'argument de `color()` est ici une constante nommée associée à un entier.

Programme (CHR.js)

```
color(X), color(Y) <=> X == red, Y == blue | color(purple)
color(X), color(Y) <=> X == red, Y == yellow | color(orange)
color(X), color(Y) <=> X == blue, Y == yellow | color(green)
```

♠

- Il est possible de factoriser les règles impliquant *brown* :

```
color(X) \ color(_) <=> X == brown | true
```

♠

Matching entre éléments de la tête

- Il est possible d'améliorer le programme en utilisant le *matching* avec les constantes directement dans les contraintes de la tête.

Programme (CHR.js)

```
color(red), color(blue) <=> color(purple)
color(red), color(yellow) <=> color(orange)
color(blue), color(yellow) <=> color(green)
color(brown) \ color(_) <=> true
```

Matching entre éléments de la tête

- Il est possible d'améliorer le programme en utilisant le *matching* avec les constantes directement dans les contraintes de la tête.

Programme (CHR.js)

```
color(red), color(blue) <=> color(purple)
color(red), color(yellow) <=> color(orange)
color(blue), color(yellow) <=> color(green)
color(brown) \ color(_) <=> true
```

- Le symbole `'_'` permet de *matcher* n'importe quelle valeur.

Matching entre éléments de la tête (suite)

- Comment enlever les contraintes en double si l'on souhaite avoir un ensemble et non un multiensemble ?

Programme (CHR.js)

```
color(X) \ color(Y) <=> X == Y | true
```

♠

Matching entre éléments de la tête (suite)

- Comment enlever les contraintes en double si l'on souhaite avoir un ensemble et non un multiensemble ?

Programme (CHR.js)

```
color(X) \ color(Y) <=> X == Y | true
```

♠

- En y intégrant le *matching*, la règle devient :

```
color(X) \ color(X) <=> true
```

♠

Le sens du *matching*

Matching	**Garde**
`c(X) <=> true`	`c(X) <=> true | true`

Le sens du *matching*

Matching	**Garde**
`c(X) <=> true`	`c(X) <=> true | true`
`c(a) <=> true`	`c(X) <=> X == a | true`

Le sens du *matching*

Matching	**Garde**
`c(X) <=> true`	`c(X) <=> true | true`
`c(a) <=> true`	`c(X) <=> X == a | true`
`c(X, X) <=> true`	`c(X,Y) <=> X == Y | true`

Le sens du *matching*

Matching	**Garde**
`c(X) <=> true`	`c(X) <=> true | true`
`c(a) <=> true`	`c(X) <=> X == a | true`
`c(X, X) <=> true`	`c(X,Y) <=> X == Y | true`
`c(X), d(X) <=> true`	`c(X), d(Y) <=> X == Y | true`

En résumé

Règles

- *head <=> Guard | Body*
  - Le corps remplace la tête si la garde est valide.
- $Head_k$ \ $Head_r$ *<=> Guard | Body*
  - Le corps remplace $Head_r$ si $Head_k$ est présent et que la garde est valide.
- *head ==> Guard | Body*
  - Le corps est ajouté lorsque la tête est présente et si la garde est valide.

- Les gardes sont optionnelles.
- Le *matching* peut être utilisé entre variables des contraintes de la tête.

7 - CHR et les variables logiques

Les variables logiques

- Les langages de programmation déclarative comme CHR intègre un type de variable : *les variables logiques*
- Elles sont similaires aux *inconnues* en mathématiques et aux variables en logiques.

Les variables logiques

- Les langages de programmation déclarative comme CHR intègre un type de variable : *les variables logiques*
- Elles sont similaires aux *inconnues* en mathématiques et aux variables en logiques.
- Une variable logique peut être soit liée (ou *instanciée*) à une valeur, soit non liée (ou *libre*, *non instanciée*).
- Ces variables ne peuvent être liées qu'une seule fois (*single assignment*).
  - Différent des variables dans un langage de programmation plus traditionnel où les anciennes valeurs peuvent être écrasées.

Les variables logiques

- Les langages de programmation déclarative comme CHR intègre un type de variable : *les variables logiques*
- Elles sont similaires aux *inconnues* en mathématiques et aux variables en logiques.
- Une variable logique peut être soit liée (ou *instanciée*) à une valeur, soit non liée (ou *libre*, *non instanciée*).
- Ces variables ne peuvent être liées qu'une seule fois (*single assignment*).
  - Différent des variables dans un langage de programmation plus traditionnel où les anciennes valeurs peuvent être écrasées.
- En *Prolog*, les variables logiques commencent par une majuscule, CHR conserve cette convention.

Unification

- L'opération principale sur les variables logiques est l'unification.

Unification

- L'opération principale sur les variables logiques est l'unification.
- En *Prolog*, il est possible d'unifier des variables logiques entre elles ou avec des termes *Prolog*.

Unification

- L'opération principale sur les variables logiques est l'unification.
- En *Prolog*, il est possible d'unifier des variables logiques entre elles ou avec des termes *Prolog*.

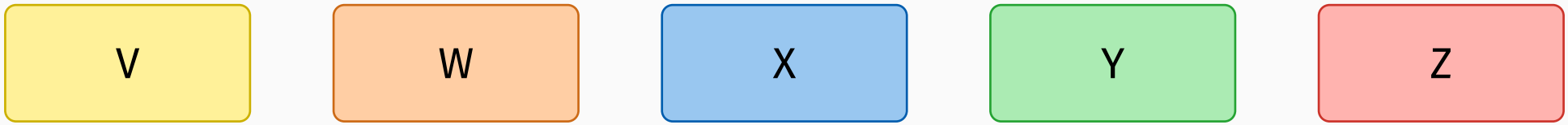
Important

- En CHR, l'unification n'est possible qu'entre variables logiques ou variable logique et constante.
- On parlera alors de classes d'équivalences entre variables logiques.
- L'unification consiste dès lors à fusionner deux classes d'équivalences de variables

Exemple d'unifications

Succession d'unifications

- Soit V, W, X, Y, et Z des variables logiques.
  - »
  - »
  - »
  - »
  - »
  - »
  - »



Exemple d'unifications

Succession d'unifications

- Soit V, W, X, Y, et Z des variables logiques.
  - $V \cong W$
  -
  -
  -
  -
  -
  -

V, W

X

Y

Z

Exemple d'unifications

Succession d'unifications

- Soit V, W, X, Y, et Z des variables logiques.
  - $V \cong W$
  - $X \cong Y$
  -
  -
  -
  -
  -

V, W

X, Y

Z

Exemple d'unifications

Succession d'unifications

- Soit V, W, X, Y, et Z des variables logiques.
  - $V \cong W$
  - $X \cong Y$
  - $V \cong Z$
  -
  -
  -
  -

V, W, Z

X, Y

Exemple d'unifications

Succession d'unifications

- Soit V, W, X, Y, et Z des variables logiques.
  - $V \cong W$
  - $X \cong Y$
  - $V \cong Z$
  - $X \cong 5$
  -
  -
  -

V, W, Z

X, Y, 5

Exemple d'unifications

Succession d'unifications

- Soit V, W, X, Y, et Z des variables logiques.
  - $V \cong W$
  - $X \cong Y$
  - $V \cong Z$
  - $X \cong 5$
  - $W \cong 3$ et $Z \cong Y$
  -
  -

V, W, Z, 3

X, Y, 5

Exemple d'unifications

Succession d'unifications

- Soit V, W, X, Y, et Z des variables logiques.
  - $V \cong W$
  - $X \cong Y$
  - $V \cong Z$
  - $X \cong 5$
  - $W \cong 3$ et $Z \cong Y \Longrightarrow$ **ÉCHEC**
  -
  -

~~V, W, Z, 3, X, Y, 5~~

Exemple d'unifications

Succession d'unifications

- Soit V, W, X, Y, et Z des variables logiques.
  - $V \cong W$
  - $X \cong Y$
  - $V \cong Z$
  - $X \cong 5$
  - $W \cong 3$ et $Z \cong Y \Longrightarrow$ **ÉCHEC**
  - *Revenons en arrière*
  -

V, W, Z

X, Y, 5

Exemple d'unifications

Succession d'unifications

- Soit V, W, X, Y, et Z des variables logiques.
  - $V \cong W$
  - $X \cong Y$
  - $V \cong Z$
  - $X \cong 5$
  - $W \cong 3$ et $Z \cong Y \Longrightarrow$ **ÉCHEC**
  - *Revenons en arrière*
  - $Z \cong Y$

V, W, Z, X, Y, 5

Intégration des variables logiques dans CHR

- Les variables logiques peuvent être utilisées comme arguments des contraintes CHR.
- L'opérateur d'unification n'est pas un élément à proprement parlé du langage CHR, c'est une contrainte built-in du langage hôte.
  - Il est possible d'en définir d'autres

Intégration des variables logiques dans CHR

- Les variables logiques peuvent être utilisées comme arguments des contraintes CHR.
- L'opérateur d'unification n'est pas un élément à proprement parlé du langage CHR, c'est une contrainte built-in du langage hôte.
  - Il est possible d'en définir d'autres
- `CHR.js` ne gère pas les variables logiques.
- *CHR+SWI Prolog* et CHR++ intègrent nativement les variables logiques.

Exemple d'utilisation des variables logiques

Programme (CHR++)

```
<chr_contraints> min(+int), res(?int)
res(X), min(Y) <=> X %= Y;;
```

♠

Exemple

```
› Logical_var<int> A;
›
›
›
  »
```

Exemple d'utilisation des variables logiques

Programme (CHR++)

```
<chr_contraints> min(+int), res(?int)
res(X), min(Y) <=> X %= Y;;
```

♠

Exemple

```
› Logical_var<int> A;
› res(A)
›
›
  »
```

res(A)

Exemple d'utilisation des variables logiques

Programme (CHR++)

```
<chr_contraints> min(+int), res(?int)
res(X), min(Y) <=> X %= Y;;
```

♠

Exemple

- Logical_var<int> A;
- res(A)
- min(3)
- »

res(3)

Exemple d'utilisation des variables logiques

Programme (CHR++)

```
<chr_contraints> min(+int), res(?int)
res(X), min(Y) <=> X %= Y;;
```

♠

Exemple

- Logical_var<int> A;
- res(A)
- min(3)
- std::cout << A << std::endl;

»

res(3)

Exemple d'utilisation des variables logiques

Programme (CHR++)

```
<chr_contraints> min(+int), res(?int)
res(X), min(Y) <=> X %= Y;;
```

♠

Exemple

- Logical_var<int> A;
- res(A)
- min(3)
- std::cout << A << std::endl;
  - » Affiche 3

res(3)

Réactivation des règles

Programme (CHR++)

```
<chr_contraints> c(?int)
c(hello) <=> printf("hello ");;
c(world) <=> printf("world ");;
```

♠

Exemple

› c(hello); c(world);
  »
›
  »
›
  »
›
  »

Réactivation des règles

Programme (CHR++)

```
<chr_contraints> c(?int)
c(hello) <=> printf("hello ");;
c(world) <=> printf("world ");;
```

♠

Exemple

› c(hello); c(world);
  » store : {} / affiche : hello world
›
  »
›
  »
›
  »

Réactivation des règles

Programme (CHR++)

```
<chr_contraints> c(?int)
c(hello) <=> printf("hello ");;
c(world) <=> printf("world ");;
```

♠

Exemple

› c(hello); c(world);
» store : {} / affiche : hello world
› c(X)
»
›
»
›
»

Réactivation des règles

Programme (CHR++)

```
<chr_contraints> c(?int)
c(hello) <=> printf("hello ");;
c(world) <=> printf("world ");;
```

♠

Exemple

- c(hello); c(world);
  - » store : {} / affiche : hello world
- c(X)
  - » store : {c(X)}
-
  - »
-
  - »

Réactivation des règles

Programme (CHR++)

```
<chr_contraints> c(?int)
c(hello) <=> printf("hello ");;
c(world) <=> printf("world ");;
```

♠

Exemple

- c(hello); c(world);
  - » store : {} / affiche : hello world
- c(X)
  - » store : {c(X)}
- c(X); X %= hello;
  - »
-
  - »

Réactivation des règles

Programme (CHR++)

```
<chr_contraints> c(?int)
c(hello) <=> printf("hello ");;
c(world) <=> printf("world ");;
```

♠

Exemple

- c(hello); c(world);
  - » store : {} / affiche : hello world
- c(X)
  - » store : {c(X)}
- c(X); X %= hello;
  - » store : {} / affiche : hello
-
  - »

Réactivation des règles

Programme (CHR++)

```
<chr_contraints> c(?int)
c(hello) <=> printf("hello ");;
c(world) <=> printf("world ");;
```

♠

Exemple

- c(hello); c(world);
  - » store : {} / affiche : hello world
- c(X)
  - » store : {c(X)}
- c(X); X %= hello;
  - » store : {} / affiche : hello
- c(X); c(world); X %= hello;
  - »

Réactivation des règles

Programme (CHR++)

```
<chr_contraints> c(?int)
c(hello) <=> printf("hello ");;
c(world) <=> printf("world ");;
```

♠

Exemple

- c(hello); c(world);
  - » store : {} / affiche : hello world
- c(X)
  - » store : {c(X)}
- c(X); X %= hello;
  - » store : {} / affiche : hello
- c(X); c(world); X %= hello;
  - » store : {} / affiche : world hello

Réactivation des règles

Programme (CHR++)

```
<chr_contraints> c(?int)
c(hello) <=> printf("hello ");;
c(world) <=> printf("world ");;
```

♠

Exemple

- c(hello); c(world);
  - » store : {} / affiche : hello world
- c(X)
  - » store : {c(X)}
- c(X); X %= hello;
  - » store : {} / affiche : hello
- c(X); c(world); X %= hello;
  - » store : {} / affiche : world hello

La réactivation

- Les contraintes sont stockées et suspendues dans le *store*.
- Une unification peut réveiller des contraintes suspendues.

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

› neq(2,2);

»

›

»

›

»

›

»

›

»

›

»

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

› neq(2,2);
» échec

›
»

›
»

›
»

›
»

›
»

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

› neq(2,2);
» échec
› neq(2,4);
»
›
»
›
»
›
»

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

```
› neq(2,2);
  » échec
› neq(2,4);
  » store : {}
›
  »
```

```
›
  »
›
  »
›
  »
```

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

› neq(2,2);
» échec
› neq(2,4);
» store : {}
› neq(A,B);
» store : {neq(A,B)}

›
»
›
»
›
»

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

› neq(2,2);
» échec
› neq(2,4);
» store : {}
› neq(A,B);
» store : {neq(A,B)}

›
»
›
»
›
»

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

› neq(2,2);
» échec
› neq(2,4);
» store : {}
› neq(A,B);
» store : {neq(A,B)}

› neq(A,B); A %= B;
»
›
»
›
»

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

› neq(2,2);
» échec
› neq(2,4);
» store : {}
› neq(A,B);
» store : {neq(A,B)}
› neq(A,B); A %= B;
» échec
›
»
›
»

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

› neq(2,2);
» échec
› neq(2,4);
» store : {}
› neq(A,B);
» store : {neq(A,B)}
› neq(A,B); A %= B;
» échec
› neq(A,B); A %= 2, B %= 2;
»
›
»

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

- neq(2,2);
  - » échec
- neq(2,4);
  - » store : {}
- neq(A,B);
  - » store : {neq(A,B)}
- neq(A,B); A %= B;
  - » échec
- neq(A,B); A %= 2, B %= 2;
  - » échec
-
  - »

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

- neq(2,2);
  - » échec
- neq(2,4);
  - » store : {}
- neq(A,B);
  - » store : {neq(A,B)}
- neq(A,B); A %= B;
  - » échec
- neq(A,B); A %= 2, B %= 2;
  - » échec
- neq(A,B); A %= 2; B %= 5;;
  - »

La contrainte d'inégalité

Programme (CHR++)

```
<chr_contraints> neq(?int,?int)
neq(X,X) <=> failure;;
neq(X,Y) <=> X.ground() and Y.ground() and X != Y | success;;
```

♠

Exemple

- neq(2,2);
  - » échec
- neq(2,4);
  - » store : {}
- neq(A,B);
  - » store : {neq(A,B)}
- neq(A,B); A %= B;
  - » échec
- neq(A,B); A %= 2, B %= 2;
  - » échec
- neq(A,B); A %= 2; B %= 5;;
  - » store : {}

En résumé

Règles

- *head <=> Guard | Body*
  - Le corps remplace la tête si la garde est valide.
- $Head_k$ \ $Head_r$ *<=> Guard | Body*
  - Le corps remplace $Head_r$ si $Head_k$ est présent et que la garde est valide.
- *head ==> Guard | Body*
  - Le corps est ajouté lorsque la tête est présente et si la garde est valide.

- Intégration des variables logiques, réveil des contraintes.

En résumé

Règles

- *head <=> Guard | Body*
  - Le corps remplace la tête si la garde est valide.
- $Head_k$ \ $Head_r$ *<=> Guard | Body*
  - Le corps remplace $Head_r$ si $Head_k$ est présent et que la garde est valide.
- *head ==> Guard | Body*
  - Le corps est ajouté lorsque la tête est présente et si la garde est valide.

- Intégration des variables logiques, réveil des contraintes.

Attention

CHR n'est pas *PROLOG* !

8 - Synthèse de cette introduction à CHR

Constraint Handling Rules (CHR)

[1991, Thom Früwirth]



- Langage de modélisation et de programmation de haut niveau à chaînage avant.
- Basé sur des contraintes et des règles :
  - *propagation*, *simplification* et *simpagation*
- Variables logiques (égalité syntactique).

- Nécessite d'être hébergé par un langage hôte.
- Un grand nombre d'implémentations (Prolog, langages impératifs et fonctionnels).
- Accompagné de sémantiques logiques pour décrire le système.
- Extensible grâce à l'ajout de contraintes built-ins.

Fermeture transitive de l'opérateur $\leq$

LEQ

réflexivité	@	$\mathrm{leq}(X, X) \iff \top$
anti-symétrie	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, X) \iff X \cong Y$
idempotence	@	$\mathrm{leq}(X, Y) \setminus \mathrm{leq}(X, Y) \iff \top$
transitivité	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, Z) \implies \mathrm{leq}(X, Z)$

♠

Exemple

› But : $\mathrm{leq}(A, B)$, $\mathrm{leq}(C, A)$ et $\mathrm{leq}(B, C)$

Fermeture transitive de l'opérateur $\leq$

LEQ

réflexivité	@	$\mathrm{leq}(X, X) \iff \top$
anti-symétrie	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, X) \iff X \cong Y$
idempotence	@	$\mathrm{leq}(X, Y) \setminus \mathrm{leq}(X, Y) \iff \top$
transitivité	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, Z) \implies \mathrm{leq}(X, Z)$

♠

Exemple

› But : $\mathrm{leq}(A, B)$, $\mathrm{leq}(C, A)$ et $\mathrm{leq}(B, C)$

$\underline{\mathrm{leq}(A, B)}$ ↦

Fermeture transitive de l'opérateur $\leq$

LEQ

réflexivité	@	$\mathrm{leq}(X, X) \iff \top$
anti-symétrie	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, X) \iff X \cong Y$
idempotence	@	$\mathrm{leq}(X, Y) \setminus \mathrm{leq}(X, Y) \iff \top$
transitivité	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, Z) \implies \mathrm{leq}(X, Z)$

♠

Exemple

› But : $\mathrm{leq}(A, B)$, $\mathrm{leq}(C, A)$ et $\mathrm{leq}(B, C)$

$\underline{\underline{\mathrm{leq}(A, B)}}$ ↦

$\underline{\mathrm{leq}(A, B)}, \underline{\underline{\mathrm{leq}(C, A)}}$ [transitivité] ↦

Fermeture transitive de l'opérateur $\leq$

LEQ

réflexivité	@	$\mathrm{leq}(X, X) \iff \top$
anti-symétrie	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, X) \iff X \cong Y$
idempotence	@	$\mathrm{leq}(X, Y) \setminus \mathrm{leq}(X, Y) \iff \top$
transitivité	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, Z) \implies \mathrm{leq}(X, Z)$

Exemple

› But : $\mathrm{leq}(A, B)$, $\mathrm{leq}(C, A)$ et $\mathrm{leq}(B, C)$

$\underline{\underline{\mathrm{leq}(A, B)}}$ ↦

$\underline{\mathrm{leq}(A, B)}, \underline{\underline{\mathrm{leq}(C, A)}}$ [transitivité] ↦

$\mathrm{leq}(A, B), \mathrm{leq}(C, A), \underline{\underline{\mathrm{leq}(C, B)}}$ ↦

Fermeture transitive de l'opérateur $\leq$

LEQ

réflexivité	@	$\mathrm{leq}(X, X) \iff \top$
anti-symétrie	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, X) \iff X \cong Y$
idempotence	@	$\mathrm{leq}(X, Y) \setminus \mathrm{leq}(X, Y) \iff \top$
transitivité	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, Z) \implies \mathrm{leq}(X, Z)$

Exemple

› But : $\mathrm{leq}(A, B)$, $\mathrm{leq}(C, A)$ et $\mathrm{leq}(B, C)$

$\underline{\underline{\mathrm{leq}(A, B)}}$	↦
$\underline{\mathrm{leq}(A, B)}, \underline{\underline{\mathrm{leq}(C, A)}}$	[transitivité] ↦
$\mathrm{leq}(A, B), \mathrm{leq}(C, A), \underline{\underline{\mathrm{leq}(C, B)}}$	↦
$\mathrm{leq}(A, B), \mathrm{leq}(C, A), \underline{\mathrm{leq}(C, B)}, \underline{\underline{\mathrm{leq}(B, C)}}$	[anti-symétrie] ↦

Fermeture transitive de l'opérateur $\leq$

LEQ

réflexivité	@	$\mathrm{leq}(X, X) \iff \top$
anti-symétrie	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, X) \iff X \cong Y$
idempotence	@	$\mathrm{leq}(X, Y) \setminus \mathrm{leq}(X, Y) \iff \top$
transitivité	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, Z) \implies \mathrm{leq}(X, Z)$

Exemple

› But : $\mathrm{leq}(A, B)$, $\mathrm{leq}(C, A)$ et $\mathrm{leq}(B, C)$

$\underline{\underline{\mathrm{leq}(A, B)}}$	↦
$\underline{\mathrm{leq}(A, B)}, \underline{\underline{\mathrm{leq}(C, A)}}$	[transitivité] ↦
$\mathrm{leq}(A, B), \mathrm{leq}(C, A), \underline{\underline{\mathrm{leq}(C, B)}}$	↦
$\mathrm{leq}(A, B), \mathrm{leq}(C, A), \underline{\mathrm{leq}(C, B)}, \underline{\underline{\mathrm{leq}(B, C)}}$	[anti-symétrie] ↦
$\underline{\underline{\mathrm{leq}(A, B)}}, \underline{\mathrm{leq}(B, A)}, B \cong C$	[anti-symétrie] ↦

Fermeture transitive de l'opérateur $\leq$

LEQ

réflexivité	@	$\mathrm{leq}(X, X) \iff \top$
anti-symétrie	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, X) \iff X \cong Y$
idempotence	@	$\mathrm{leq}(X, Y) \setminus \mathrm{leq}(X, Y) \iff \top$
transitivité	@	$\mathrm{leq}(X, Y), \mathrm{leq}(Y, Z) \implies \mathrm{leq}(X, Z)$

Exemple

› But : $\mathrm{leq}(A, B)$, $\mathrm{leq}(C, A)$ et $\mathrm{leq}(B, C)$

$\underline{\underline{\mathrm{leq}(A, B)}}$ ↦

$\underline{\mathrm{leq}(A, B)}, \underline{\underline{\mathrm{leq}(C, A)}}$ [transitivité] ↦

$\mathrm{leq}(A, B), \mathrm{leq}(C, A), \underline{\underline{\mathrm{leq}(C, B)}}$ ↦

$\mathrm{leq}(A, B), \mathrm{leq}(C, A), \underline{\mathrm{leq}(C, B)}, \underline{\underline{\mathrm{leq}(B, C)}}$ [anti-symétrie] ↦

$\underline{\underline{\mathrm{leq}(A, B)}}, \underline{\mathrm{leq}(B, A)}, B \cong C$ [anti-symétrie] ↦

$A \cong B, B \cong C$

9 - Quel système CHR choisir ?

Principaux systèmes existants

- Des implémentations pour différents langages hôtes.
- Systèmes pour des langages de la famille de Prolog :
  - SICStus, Yap, Eclipse, XSB, hProlog, HAL, SWI ;
  - système de référence pour SWI-Prolog *[K.U Leuven]* ;
  - les plus complets (CHR^{v}).

Principaux systèmes existants

- Des implémentations pour différents langages hôtes.
- Systèmes pour des langages de la famille de Prolog :
  - `SICStus`, `Yap`, `Eclipse`, `XSB`, `hProlog`, `HAL`, `SWI` ;
  - système de référence pour `SWI-Prolog` *[K.U Leuven]* ;
  - les plus complets (CHR^{v}).
- Systèmes pour des langages fonctionnels :
  - `Haskell` : axé sur le parallélisme.

Principaux systèmes existants

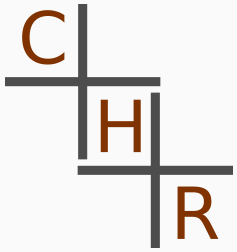
- Des implémentations pour différents langages hôtes.
- Systèmes pour des langages de la famille de Prolog :
  - `SICStus`, `Yap`, `Eclipse`, `XSB`, `hProlog`, `HAL`, `SWI` ;
  - système de référence pour `SWI-Prolog` *[K.U Leuven]* ;
  - les plus complets (CHR$^{\vee}$).
- Systèmes pour des langages fonctionnels :
  - `Haskell` : axé sur le parallélisme.
- Systèmes pour des langages impératifs (les plus performants) :
  - `Java` : `JaCK` (toolkit), `DJCHR` (DSL), `JCHR` ;
  - `C` : `CCHR` ;
  - `JavaScript` : `CHR.js` (absence de variables logiques) ;
  - absence de la gestion du non-déterminisme *don't know*.

Principaux systèmes existants

- Des implémentations pour différents langages hôtes.
- Systèmes pour des langages de la famille de Prolog :
  - `SICStus`, `Yap`, `Eclipse`, `XSB`, `hProlog`, `HAL`, `SWI` ;
  - système de référence pour `SWI-Prolog` *[K.U Leuven]* ;
  - les plus complets (CHR^{v}).
- Systèmes pour des langages fonctionnels :
  - `Haskell` : axé sur le parallélisme.
- Systèmes pour des langages impératifs (les plus performants) :
  - `Java` : `JaCK` (toolkit), `DJCHR` (DSL), `JCHR` ;
  - `C` : `CCHR` ;
  - `JavaScript` : `CHR.js` (absence de variables logiques) ;
  - absence de la gestion du non-déterminisme *don't know*.
- Système développé au LERIA pour C++ : CHR++.

CHR++

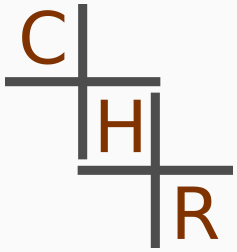
Un système CHR pour C++



- CHR++ est un système CHR au-dessus de C++.
  - https://gitlab.com/vynce/chrpp
- Implémente l'intégralité de CHR.
  - *contraintes, règles, variables logiques.*
- Intègre le non-déterminisme *don't know* ($\text{CHR}^{\vee\!\vee}$) :
  - *variables logiques et stores de contraintes* backtrackables.
- Composé d'un compilateur et d'une partie runtime.

CHR++

Un système CHR pour C++



- CHR++ est un système CHR au-dessus de C++.
  - https://gitlab.com/vynce/chrpp
- Implémente l'intégralité de CHR.
  - *contraintes, règles, variables logiques.*
- Intègre le non-déterminisme *don't know* ($CHR^{\vee\!\vee}$) :
  - *variables logiques et stores de contraintes* backtrackables.
- Composé d'un compilateur et d'une partie runtime.

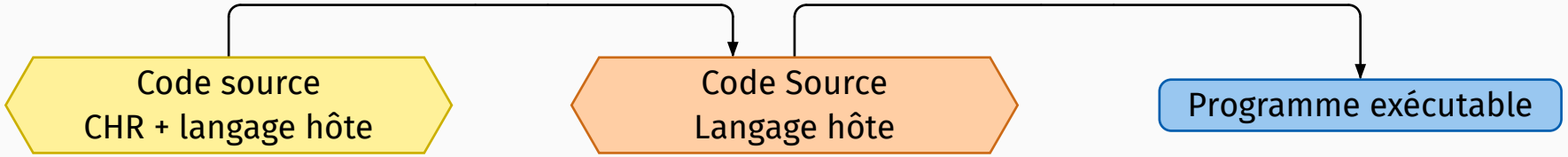


Schéma de compilation d'un programme CHR++

Fichier test.chrpp

```
<CHR name="MonPrg">
<chr_constraint> res(+int)
r1 @ res(_) <=> success();;
</CHR>
int main()
{
    ...
}
```

Schéma de compilation d'un programme CHR++

`chrppc test.chrpp`

Fichier test.chrpp

```
<CHR name="MonPrg">
<chr_constraint> res(+int)
r1 @ res(_) <=> success();;
</CHR>
int main()
{
    ...
}
```

Schéma de compilation d'un programme CHR++

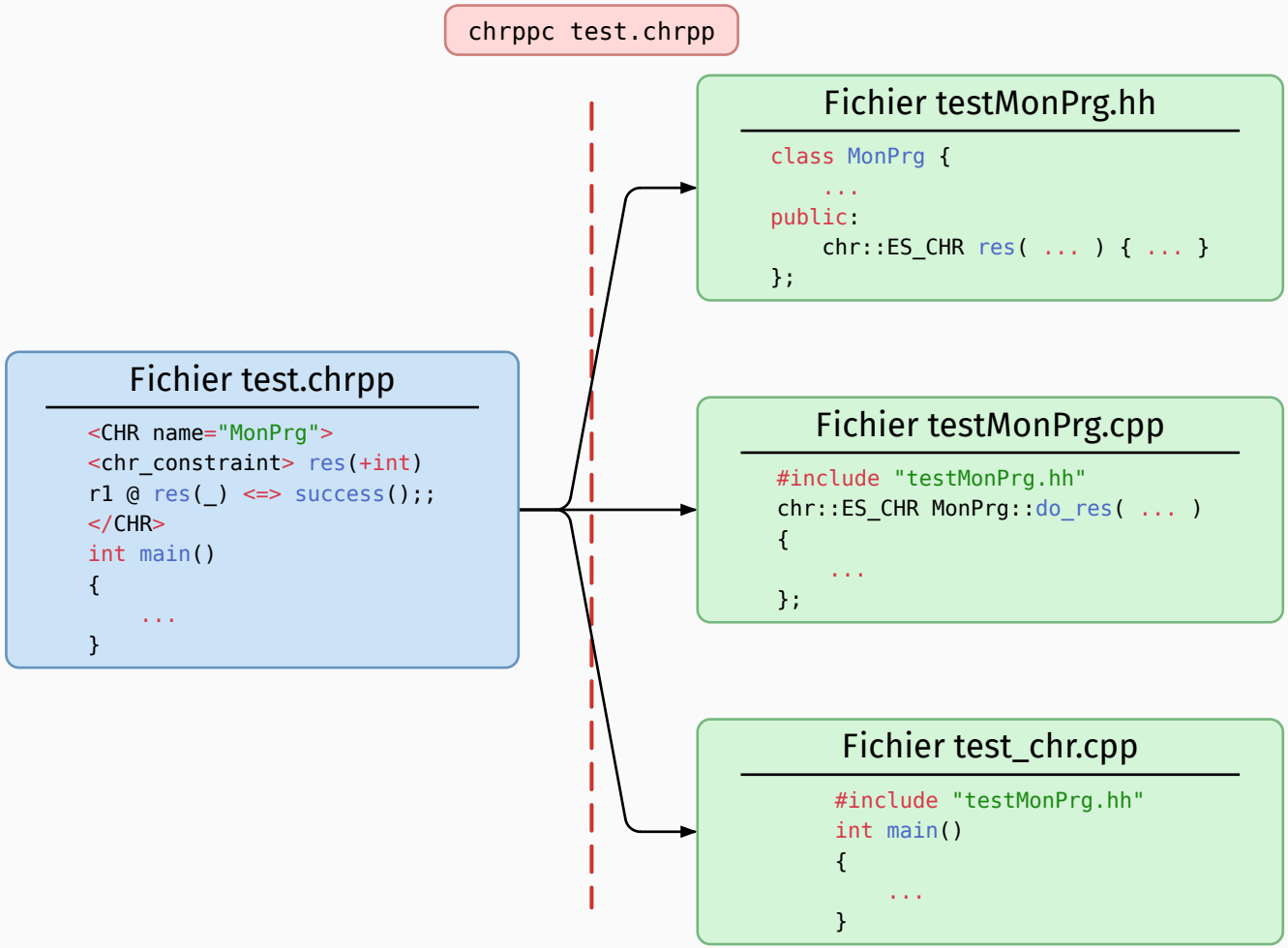


Schéma de compilation d'un programme CHR++

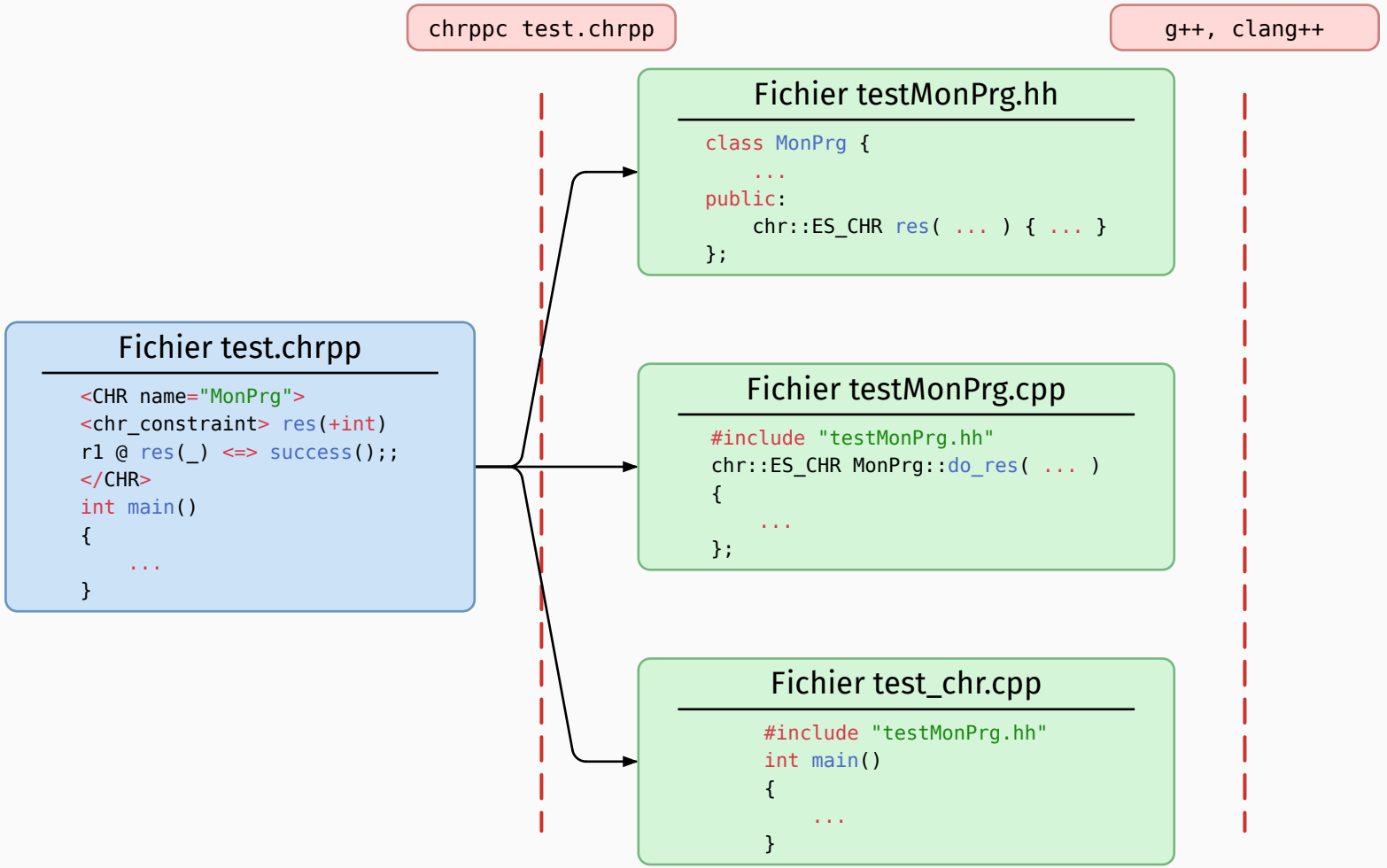
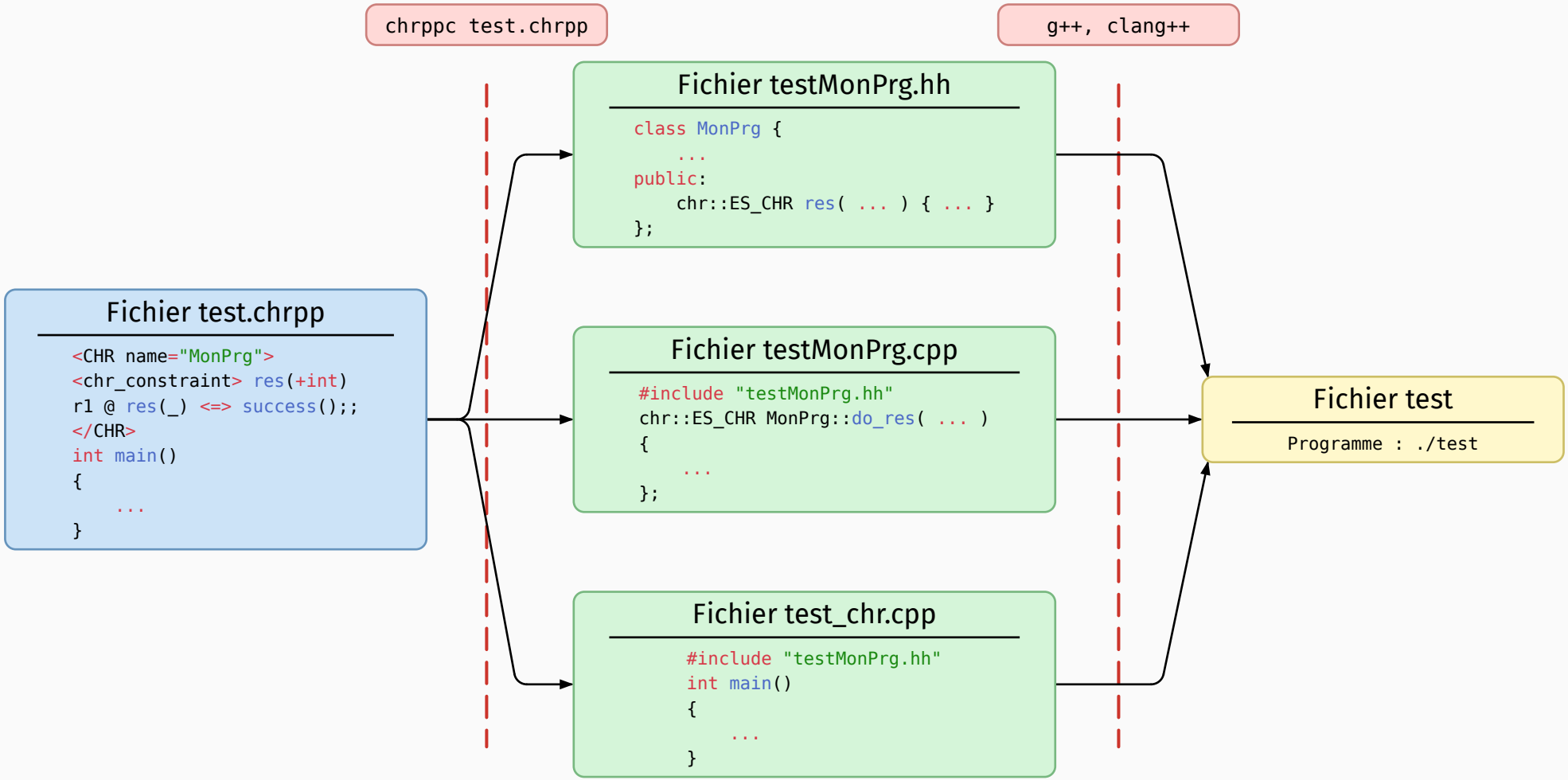


Schéma de compilation d'un programme CHR++



Compilation d'un programme CHR++

Application au programme de calcul du PGCD

```
// Fichier nommé : gcd.chrpp
#include <iostream>
#include <chrpp.hh>
```

Compilation d'un programme CHR++

Application au programme de calcul du PGCD

```
// Fichier nommé : gcd.chrpp
#include <iostream>
#include <chrpp.hh>

<CHR name="GCD">
    <chr_constraint> gcd(+int), res(?int)
    gcd1 @ gcd(0) <=> success();;
    gcd2 @ gcd(N) \ gcd(M) <=> N <= M | gcd(M-N);;
    res  @ gcd(N) \ res(M) <=> M %= N;;
</CHR>
```

Compilation d'un programme CHR++

Application au programme de calcul du PGCD

```
// Fichier nommé : gcd.chrpp
#include <iostream>
#include <chrpp.hh>

<CHR name="GCD">
    <chr_constraint> gcd(+int), res(?int)
    gcd1 @ gcd(0) <=> success();;
    gcd2 @ gcd(N) \ gcd(M) <=> N <= M | gcd(M-N);;
    res  @ gcd(N) \ res(M) <=> M %= N;;
</CHR>

int main()
{
  chr::Logical_var<int> Res;
  auto space = GCD::create();
  space->gcd(27); space->gcd(24); space->res(Res);
  std::cout << "gcd(27,24) = " << Res << std::endl;
  return 0;
}
```

Compilation d'un programme CHR++

Application au programme de calcul du PGCD

```
// Fichier nommé : gcd.chrpp
#include <iostream>
#include <chrpp.hh>

<CHR name="GCD">
    <chr_constraint> gcd(+int), res(?int)
    gcd1 @ gcd(0) <=> success();;
    gcd2 @ gcd(N) \ gcd(M) <=> N <= M | gcd(M-N);;
    res  @ gcd(N) \ res(M) <=> M %= N;;
</CHR>

int main()
{
  chr::Logical_var<int> Res;
  auto space = GCD::create();
  space->gcd(27); space->gcd(24); space->res(Res);
  std::cout << "gcd(27,24) = " << Res << std::endl;
  return 0;
}
```

- Appel de `chrppc gcd.chrpp` :
  - `gcdGCD.hh`, `gcdGCD.cpp` et `gcd_chr.cpp`.

Compilation d'un programme CHR++

```
// Fichier nommé : gcd.chrpp
#include <iostream>
#include <chrpp.hh>

<CHR name="GCD">
    <chr_constraint> gcd(+int), res(?int)
    gcd1 @ gcd(0) <=> success();;
    gcd2 @ gcd(N) \ gcd(M) <=> N <= M | gcd(M-N);;
    res  @ gcd(N) \ res(M) <=> M %= N;;
</CHR>

int main()
{
  chr::Logical_var<int> Res;
  auto space = GCD::create();
  space->gcd(27); space->gcd(24); space->res(Res);
  std::cout << "gcd(27,24) = " << Res << std::endl;
  return 0;
}
```

Application au programme de calcul du PGCD

- Appel de `chrppc gcd.chrpp` :
  - `gcdGCD.hh`, `gcdGCD.cpp` et `gcd_chr.cpp`.
- Analyse et optimisation :
  - late storage ;
  - contraintes jamais stockées ;
  - stores indexées ;
  - réarrangement des têtes et des gardes ;
  - …

Compilation d'un programme CHR++

Application au programme de calcul du PGCD

```
// Fichier nommé : gcd.chrpp
#include <iostream>
#include <chrpp.hh>

<CHR name="GCD">
    <chr_constraint> gcd(+int), res(?int)
    gcd1 @ gcd(0) <=> success();;
    gcd2 @ gcd(N) \ gcd(M) <=> N <= M | gcd(M-N);;
    res  @ gcd(N) \ res(M) <=> M %= N;;
</CHR>

int main()
{
  chr::Logical_var<int> Res;
  auto space = GCD::create();
  space->gcd(27); space->gcd(24); space->res(Res);
  std::cout << "gcd(27,24) = " << Res << std::endl;
  return 0;
}
```

- Appel de `chrppc gcd.chrpp` :
  - `gcdGCD.hh`, `gcdGCD.cpp` et `gcd_chr.cpp`.
- Analyse et optimisation :
  - late storage ;
  - contraintes jamais stockées ;
  - stores indexées ;
  - réarrangement des têtes et des gardes ;
  - …
- Compilation des fichiers C++ :
  - `g++` ou `clang++`.
- Exécution du programme `./gcd` :
  - Résultat : `gcd(27,24) = 3`.